

提示词工程 [02-Raw/26生成式软件工程/NJU]

原视频: <https://www.bilibili.com/video/BV1CQt365EzW> · 课程合集:
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

全课主线

讲者想让听众相信: 自然语言提示词是人和 AI 沟通的核心接口, 但真正要优化的不是聊天框里的一句话, 而是 agent 看到的整个线性上下文, 因此“提示词工程”实质是上下文工程。要进一步理解模型如何按 next token prediction、self-attention、CoT 和 verifier 遵循指令, 把模糊 intent 补成明确设计、spec 和可验证条件; 当前 AI 不是超能力, 人仍须有概念、提出正确问题、设计引导和 verify, 并用 AI 加速学习与协作。最终应把确定的具体任务交给 AI, 把高不确定性探索和最优判断留给人, 并在 AI 关键奇点前思考如何防止危险后果。

1. 起点是自然语言提示词成为人与 AI 沟通的核心接口; 但 agent 在用户开口前已经看到 system prompt、AGENTS.md、skills、工具调用与结果等长线性上下文, 聊天框里的 user message 只是最后追加的一段。(第 1 部分)
2. 因此, 优化提示词不能只改一句指令, 而要把模型行为理解为由整个上下文决定; “提示词工程”应被理解为上下文工程。模型按 next token prediction 推进, 后训练强化了长上下文中的指令遵循, 但早期指令仍可能因截断或压缩而丢失, 所以需要重新设计上下文。(第 2 部分)
3. 上下文进一步被讲者拆成分层注入的长序列: 全局、项目级、agent 目录的 AGENTS.md, 常驻的 skill 索引, 按需加载的完整 SKILL.md, 工具定义、会话历史和环境变量; 上下文工程就是安排这些。讲者由此建议, 遇到一点麻烦就立即用 AI, 并让 agent 自查上下文, 同时把 AI 当作费曼学习法的老师/学生。(第 2、3、4 部分)
4. 对 agent 的观察继续推进到安全与自动化: YOLO 模式快但危险, 沙箱更安全却会打断流程; 风险判定不能只看命令表面, Codex 用模型评级加静态分流, 安全评估 prompt 可打印、可改写, 行为变化可作证据。(第 4、5、6 部分)
5. 写好的上下文要回到软件工程大流程: 把 intent、spec、架构与 implementation 中的一部分切片塞给模型, 并在脑中目标与给模型的指令之间找 gap; 模糊 intent 会让模型

漂移并误以为完成，完整设计、spec 和机器可验证条件才能控制漂移。(第 6、7、8 部分)

6. 但直接生成还会带来 AI 味与同质化，因为模型不受控时品位固定、会朝固定方向发散；所以生成式软件工程的核心是明确意图，用上下文工程约束发散，并通过接触多样性更精确表达自己想要什么。(整理者串联)(第 8、9、10 部分)
7. 要更好地下指令，必须理解模型如何遵循指令：模型是概率分布，每次只做有限的 next token prediction；CoT 把上下文当作 append-only 的纸笔，先写出候选解再验证；早期幻觉来自模型被激励必须输出，强化学习后则学会必须给对。(第 10、11、12 部分)
8. 因此 prompt 的关键是给模型一个明确的 verifier；有明确 verifier 的部分更容易交给 AI，教学等任务难在“好”没有可形式化的 verifier。Self-attention 让模型按需分配注意力，CoT token 会重新选择关注点；指令越多越抢注意力，早期角色扮演式 prompt 是在 hack self-attention，后训练/RLHF 后更依赖明确 verifier，但纠缠会污染上下文，所以是 trade-off。(第 13、14、15、16 部分)
9. 这种注意力与上下文污染解释了读论文时模型会延续作者逻辑、AI 容易当墙头草；所以要把 AI 与“我”对齐，用人类知识边界或学生当前水平做 grounding，把水平写入 memory，先达成共识再评增量；让 AI 做可 verify 的一步推理，而审稿核心是逻辑是否断开、是否增加人类新知识。AI 写/审循环会导致 slop，valid paper 不等于值得读。(第 17、18、19、20、21 部分)
10. 学习与使用上，讲者主张用新 fact 对齐过去的我，扩大“我的世界”；AI 能加速扩展边界，但边界靠实践失败长出。用好大模型要先会遵循指令，再处理真正 long horizon；真正 long horizon 带未知，现阶段应让 AI 做明确局部任务如 PA2，不让它做长 planning；从零做产品容易，后续维护会翻车，而大型、高质量、维护良好的软件系统反而给 AI 好上下文。(第 21、22、23、24 部分)
11. 课程核心是：今天还没有超能力 AI，人仍要有概念、知道上下文里有什么以及每个东西会让 AI 产生什么结果；要用软件工程方法给指令、监控 AI、定义好的 verify，把正确简练的提示词持续放进上下文。错误目标如“证明 A 方法有用”会把模型变成 reward hacker，纠正要从真实工具和 SOTA 开始；与 AI 协作是在盯着纠正和给更多 token 之间权衡，长上下文指令遵循会下降，但最后一条具体指令仍常可遵循，所以把确定的具体活交给 AI，把高不确定性探索和最优判断留给自己。(第 25、26、27 部分)
12. 最后，验证容易、找到满足条件的解难，模型为完成任务只能有限搜索；直接 next token 会趋同，系统性拆分、枚举并用 subagent 检查可实现算力换智力，代价是烧 token。个体 bias 使群体出现 $1+1>2$ ，但若有人获得无限算力与智力，可能形成墙内超人、墙外动物；AI 关键奇点临近，先要思考如何防止。(第 28、29 部分)

讲者的号召是：把提示词工程当作上下文工程来实践，明确意图、给出好的 verifier，守住人的概念、领域知识与高收益探索，并提前思考如何防止 AI 关键奇点带来的危险。

第 1 部分

仙1-107老师



仙1-107老师

Prompt Engineering

居然以一个非常意外的方式上热搜了

受到了朋友们的问询 🤔

- Anyway, 你们懂我不会真的让你们退学的

- 但如果这么说不行, 你们真的听不进 (虽然说了也听不进)

- 而且 Token 也没那么贵, 这一百万步讲, 还有 ¥40 的豆包

(含 GLM, DeepSeek, ...)

提示词

[[搜索文献, 确认语言模型 “提示词” 的由来]]

生成式软件工程的 “生成” 是由自然语言驱动的。对于 Agent 来说, 自然语言不仅是输入, 还分散在各个地方, 但我们还是习惯从 ChatGPT 时代的术语 “提示词” (prompt), 以下都是提示词:

- Harness 内部 (system prompt, goal 模式注入等)

- AGENTS.md 等

- 加载的 skills

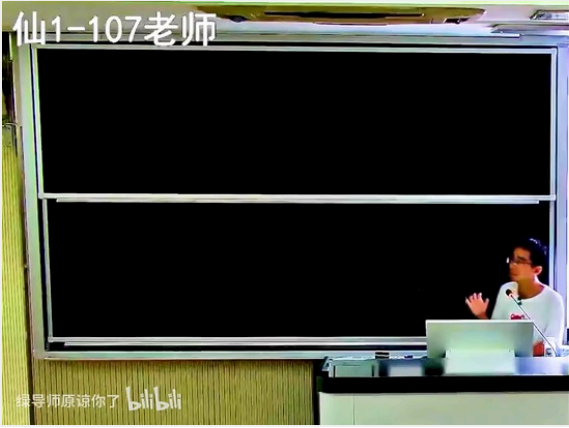
- 你给的指令

再加上大量的工具/skills 调用和输出, 构成了模型的**上下文**

也就是 Pr[token | context] 的 context。~Claude Opus 4.5

g2/lect/index.md 18,1 Top

仙1-107老师



仙1-107老师

“提示词” 从哪里来?

To exit full screen, press and hold [Esc]

ChatGPT 普及了 Prompt, 但没有发明它



◆ 2019 年 GPT-2 的发布页面已经写下 “prompting the trained model”: 输入文本从 “开头” 变成了控制模型续写的条件。GPT-2, 2019

◆ 2020 年 GPT-3 把任务说明和 few-shot 示例都称为 prompt; 2021 年, “prompt programming” 和 “prompt-based learning” 成为研究对象。

1 / 7

仙1-107老师



仙1-107老师

提示词不止在对话框

Agent 看见的是一整个上下文

以下都是提示词:

• Harness 内部: system prompt, goal 模式注入等

• AGENTS.md 等持久指令

• 加载的 skills

• 你给的指令

• 工具/skills 的调用和输出

◆ 不过, 不是所有 context 都是指令: 代码、历史消息、工具返回也可能只是事实、状态或样例。

Pr[token | context]

2 / 7

自然语言提示词是人和 AI 沟通的核心接口

- 讲者认为，自然语言提示词基本上就是人和机器沟通的“唯一接口”。

- 从 ChatGPT 时代开始，大家在聊天窗里给机器一个指令，也就是 **prompt**，AI 返回文字、代码，甚至图片；这些内容都可能通过 prompt 来生成。
- 在 ChatGPT 之前，人们并没有在这个意义上讨论 **prompt**；ChatGPT 之后，prompt 才成为一个新词。

讲者的亲历：让 ChatGPT 讲 inline assembly，第一次感到“时代要变了”

- 讲者每年上操作系统课。2023 年，也就是 ChatGPT 在 2022 年底刚发布时，他试图让 ChatGPT 帮他讲 **inline assembly**。
- 讲者插话：现在应该已经没人写 inline assembly 了，它成了“传统艺能”。
- ChatGPT 解释得出奇地好。讲者判断，这大概不在它的训练语料之内；他还故意构造了一段“地球人应该不会那样实现”的 inline assembly，但 ChatGPT 做得相当好，虽然确实不完美。
- 那一刻讲者觉得：时代可能要变了。只是当时还没有今天这么清楚地感受到，时代会变，以及具体会怎样变。

聊天框里打的字，只是 prompt 的一小部分

- 常见理解是：prompt 就是聊天框里那段话，比如“帮我生成一个游戏”“帮我把这个作业做完”。
- 讲者纠正：提示词不只在对话框里。**agent 看到的是一整个 agent 上下文**。
- 在你和 agent 聊天之前，它已经能够看到一个 **system prompt**，这也是提示词。

Agent 在开口前已经加载了一长串线性上下文

- 在 system prompt 之后，它还会加载很多东西：例如 **AGENTS.md**，还可能加载 **skills**；AGENTS.md 里还会要求它再去读别的文件。
- 到 agent 第一次停下来和你对话时，它里面已经有很多很多内容了。
- 最开始的 system prompt 可能来自你的 **agent harness**，例如 Codex、Claude Code 这类工具的内建内容。
- 除此之外，所有**工具调用和工具调用结果**，实际上也会出现在上下文里。
- 这个上下文是**线性的**。直到有一天，user 才在里面说：“给我做一个游戏吧。”

因此 prompt 很长，今天的 agent 能力显得“离谱”

- 因为 agent 能看到非常长的提示词/上下文，所以今天 agent 的能力非常离谱。

- 例子：在最早的 system prompt 里可以要求它用中文回答。讲者为了上课效果就这样做；哪怕他用英文写 prompt，user message 是英文，输出仍然是中文。
- 讲者说：“你们会看到这个行为。”

结论：优化提示词，实际上是在做上下文工程

- 讲者说，当我们说“优化提示词”时，不仅仅是在给它一个指令。
- 当我们说“提示词工程”时，实际上表达的是一个叫上下文工程的东西。
- 逻辑链：
 1. 大众以为 prompt = 聊天框里的那句话。
 2. 实际 agent 看到的是 system prompt → AGENTS.md/skills → 工具调用及结果 → 你的 user message 的完整线性上下文。
 3. 你的那句话只是这条长上下文最后追加的一段。
 4. 讲者由此说：优化提示词不仅仅是给它一个指令。
 5. 所以“提示词工程”实质上就是上下文工程。

第 2 部分



从提示词工程到上下文工程

工程化模型在每一步真正看见什么

- “提示词工程”关心一句话怎么写；“上下文工程”关心每次决策前，哪些 token 以什么来源、顺序和作用域出现。
- ♦ 目标是让 working set 足够小、足够新、带来源、可验证；上下文窗口装得下，不等于注意力用得好。
- 到 Claude Opus 4.5 这一代，模型靠强化学习获得了极强的指令遵循能力，直接输出命令和工具调用，持续向目标推进。
 - ♦ 模型只看到 token 序列；system/user/tool 的边界、优先级和权限，都是 Harness 放进上下文并在外部执行的。



优化提示词不是只改一句提示，而是在做上下文工程

讲者主张：不能只给模型“一条指令”；当我们说优化提示词、做提示词工程时，实际表达的是上下文工程，也就是管理整个模型的上下文。

讲者的逻辑链：

1. 模型做的是 next token prediction。
2. 它每一次预测下一个 token，这个 token 可以是调用工具，也可以只是输出一个字符。
3. 它会不停向后推进任务的执行。→ 所以真正影响模型行为的不是一两个提示词，而是整个上下文。讲者结论：提示词工程应理解为上下文工程。

从 Claude Opus 4.5 这一代起，长上下文中的指令遵循被强化学习与后训练显著拉长

讲者感觉，大概从 Claude Opus 4.5 这一代开始，Anthropic 发现模型靠强化学习和后训练，突然学会非常强的指令遵循能力。

具体表现（讲者说）：

- 上下文可以非常长，接近百万。
- 即使指令在很后面，模型依然能遵循你的指令。
- 它还能一定程度上看到前面的东西。

讲者的链条：Anthropic 发现模型通过强化学习 + 后训练获得强指令遵循 → 长上下文里也能在很后面遵循指令 → 于是可以把指令放在很早的 prompt 里，并观察它能否维持很久。

“喵”丢了：早期指令失效，说明上下文被截断/压缩，需要重设上下文工程

讲者举了一个著名梗：你可以在最早的 prompt 里加一句要求——“你每说一句话都要在后面加一个喵”。然后你和它工作、工作、工作，某一天突然发现，它跟你聊天时那个“喵”丢了。

讲者解释：这不管是因为上下文溢出被截断，还是被压缩等等，某种原因，反正它开始不能遵循指令了。这时你就需要重新想一个办法。

讲者认为：这就是大家在工程实践中发现的各种各样有趣的上下文工程，也就是提示词工程的方式。

链条： 早期指令依赖完整上下文 → 上下文被截断/压缩 → 早期指令丢失 → “喵”不再出现 → 模型开始不能遵循指令 → 需要重新设计上下文。

讲者最重要的建议：觉得有一点麻烦，就立即用 AI

讲者说，他上这门课希望告诉大家最重要的一件事；他提到“没有 token 就应该退学”。他想传达的是：所有事情都应该立即开始尝试 AI 给我们的能力——任何时候你觉得有一点麻烦，你都可以立即行动。

讲者认为，**AI 是非常耐心的老师**。他回忆上大学时：有东西不会，可能室友会；但和室友沟通成本很高，还会担心是不是在室友面前暴露了自己很菜，这可能是他不想告诉室友的秘密。讲者说，这也正常。

但 AI 老师不会嫌弃你，而且你可以任意程度地追问。

agent 能检查自己的上下文，所以立刻让它自查

讲者接着前面说的概念：我们的 AI agent 有一个很长的上下文，它有 **system prompt**，有各种各样的东西。然后他说，下一秒我就可以立即打开一个 agent，直接问它：

老师上课的时候说，agent 的上下文里不仅是一两个提示词，你能检查一下你的上下文里都有些什么吗？

讲者说：

- agent 其实可以自己检查自己；
- 甚至可以检查自己的日志；
- 这 agent 经常给你惊喜：当你问出一个问题，它的解决手段会出乎你的意料。

原因链（讲者）：agent 正好可以看到自己的上下文 → 根据上课所说，它是 next token prediction，能看到所有东西 → 所以它应该能读出来，除非有明确指令禁止。

第 3 部分



仙1-107老师

导师原谅你了 bilibili

```
SKILL.md
Took 0.0s

Thinking...

你的问题正好可以“实测”——我直接翻了一下自己的上下文。所谓“一两个提示词”只是冰山一角，我的上下文实际由这些部分组成：

1. 系统提示词 (System Prompt)
来自 pi 本身 (~/.local/share/pi-node/.../README.md 里的编码助手行为规范)：工具使用指南、回复要简洁、如何调用 bash/edit 等。这是“主提示词”，但只占很小一部分。

2. 用户级指令 /home/jyy/.pi/agent/AGENTS.md
- 课堂演示风格：简洁、建设性，用中文回答、结尾必须写 >
Recap: 引用块、git 提交遵循现有风格。

3. 项目级指令 /home/jyy/Projects/GenSE/AGENTS.md
- 身份设定：2026《生成式软件工程》教学辅助 Agent；使用者是薛
3. 项目级指令 /home/jyy/Projects/GenSE/AGENTS.md
- 身份设定：2026《生成式软件工程》教学辅助 Agent；使用者是薛
发布教程：
- 领域知识：火山多模态 API 的地址、API key 存放位置 (~/.local/workspace/api_keys.txt)、"不要过度追求语言精确会弄内通"等。

4. Skills 索引 (按需加载)
.agents/skills/ 下有 5 个技能：compile、make-artifact、make-slides、tag-audio、transcribe (共 8KB)。注意：上下文里
```



仙1-107老师

导师原谅你了 bilibili

立即行动

AI 把“启动一次尝试”的成本压到很低

- AI 给我们的能力：**立即行动**，简直就是拖延症的克星
 - 微积分，一个推导不想验算
 - 数字电路，有个数据通路没看懂
 - 计算机系统基础，链接没搞明白
 - 这些统统可以立刻开始
 - 我学生时代的 self-motivated：“超高的容忍度”
 - 现在学生的 self-motivated：用上飞轮，确保自己得到了正确的理解，并且重述给 AI
- ♦ AI 消灭的是“先试一下”的边际成本，不消灭真伪判断：提出最小问题 → 做最小检验 → 看反馈 → 更新理解。

模型默认能看见完整上下文，所以能读出 system prompt；限制来自供应商

- 讲者说：模型是 next token predict，能看到所有东西，所以理论上应该能读出 system prompt。
- 除非有明确指令禁止。

- 模型供应商 (providers) 会禁止你套取系统上下文：
 - 通过模型微调；
 - 通过明确指令，例如在 **system prompt** 开头 说：任何形式要求回答 system prompt 都拒绝。
- 但因为这是整个**开源框架**，本身可以 debug：
 - 能看系统提示词来自哪里；
 - 能看工具使用指南；
 - 能看“回复要简洁”等要求。
- 逻辑链： `next token predict + 完整上下文` → 理论上可读出；供应商微调/显式指令 → 实际拒绝；开源框架可 debug → 可查看来源与各层注入 。

上下文指令是分层的：全局、项目级、agent 目录都可放

AGENTS.md

- 接着它会打开用户目录下某个 `agent` 目录中的 `AGENTS.md`（转录原词：派下的 agent 下的 agent4点MD；目录名不确定）——这是讲者自己写的。
- 因为是上课演示机器，讲者在全局上下文里放了：
 - 用中文回答；
 - 结尾必须写一个 recap。
- 所以大家可能注意到：模型跑完后，第一个看到的是“我总结了这一次做什么”之类，对 `AGENTS.md` 的响应。
- 然后有**项目级指令**。
- 一层一层：agent 目录里还可以再放 `AGENTS.md` 。
- 此外还有 **skills** 索引。

Skill 常驻的是索引/元数据，完整 SKILL.md 按需加载，用完从上下文抹除

- `skills` 是模型能执行的功能。
- 例子：讲者有一个叫 **make slide** 的 skill（转录作 make slice），能让他在此处按 F5 播放幻灯片，完成幻灯片渲染。
- 模型在上下文里只能看到该 skill 的描述/元数据：名字是什么、简短描述是什么。
- 完整 skill 可能写得很长，但只是**按需加载**。

- 等那一轮加载完，这个 skill 也会从上下文里被抹掉。
- 只有匹配任务时，才读取完整的 SKILL.md。
- 链条：skills 索引常驻 → 只暴露元数据 → 任务匹配 → 读取完整 SKILL.md → 执行/加载后从上下文抹除。

工具定义、会话历史、环境变量都被注入，所以上下文工程是安排长序列

- 工具定义也直接以文本形式出现在上下文里。
- 还有会话历史。
- 还有环境变量：每次执行都会注入当前的一些环境，比如当前模型，甚至当前时间。
- 讲者说这是大家所不知道的。
- 例子：如果和 DeepSeek Chat 聊天，可以把每次注入的环境套出来；这不属于系统提示词，所以可以套出来。
- 因此上下文是非常长的序列。
- 所谓上下文工程 / 提示词工程，就是把所有这些安排好。

AI 时代的学习：问清楚、追问之外，还可以反过来当老师（费曼学习法）

- 讲者说，刚才其实演示了 AI 时代应该怎么学习。
 - 不仅可以像刚才那样把问题问清楚；
 - 在不明白的时候追问；
 - 还可以把它反过来。
 - 大家可能听过费曼学习法：怎么判定自己是否理解知识？反过来你当老师。
 - 讲者认为，在学生时代这样做代价非常大：因为你要当老师，就必须有一个听众来检验你。
 - 听众很可能是室友；室友可能不想听你啰嗦，可能不感兴趣，可能对这个了解得太清楚了。
-

第 4 部分



费曼学习法：让 AI 当听众

不是让 AI 再讲一遍，而是主动暴露缺口



◆ 先合上资料，用自己的话讲：卡住的地方信息量最大。AI 的角色是追问、找反例和验算，查证后再讲一遍。

◆ 这套飞轮有实证支点：[self-explanation](#) 与 [retrieval practice](#)。

5/7



签名，模型靠它们才知道“能做什么”。

6. 会话历史（持久化的上下文）

~/pi/agent/sessions/ 下的 .jsonl 文件（当前：2026-09-01T10-17-05-223Z_...jsonl）。里面不只是对话，还有事件元数据：模型切换、thinking level、每次调用的 token 用量和费用（看这页：input:107, cacheRead:2964←说明系统提示词+项目指令等前缀被缓存了，实际只付了增量 token 的钱）。

7. 环境变量（PI_*）

PI_MODEL=deepseek-v4-flash, PI_REASONING_LEVEL=high, PI_SESSION_ID, cwd 等，注入到提示词里让 agent 知道“我是谁、在哪里”。

一句话总结：提示词只是“种子”，真正的上下文 = 多级指令文件（用户级/项目级）+ 技能索引 + 工具定义 + 会话历史 + 环境信息的复合体，而且带缓存和按需加载机制。

Recap: 实际检查了 pi agent 的上下文构成——系统提示词、用户级与项目级 AGENTS.md 多级指令、按需加载的 skills 索引、工具 schema、jsonl 会话历史（含 token 用量元数据）以及 PI_* 环境变量；并观察到 cacheRead 机制证明前缀被缓存复用，印证了“上下文不只是提示词”的说法。

~/Projects/GenSE (main)
t3.0k 42.1k R16k CH90.5% \$0.001 0.8k/1.0M (auto) deepseek-v4--

没有配合的听众时，把 AI 变成费曼学习法的“老师”和“学生”

- 讲者先指出传统 费曼学习法 的现实障碍：你要讲给别人听，但那个听众很可能是你的室友；室友可能不想听你啰嗦、对这个不感兴趣、已经了解得太清楚，或者单纯不想互相帮助。
- 讲者给的替代方案：这时可以把 AI 当老师，向它请教，反复请教，把自己弄懂。
- 自认为弄懂之后，再反过来自己当老师：让 AI 扮演学生，而且“假装什么都不知道”，你把概念讲给它听。
- 讲者给出的双向校验链：
 - 我把 AI 当老师 → 向它请教 → 弄懂概念；
 - 我自认为弄懂 → 反过来当老师 → 讲给 AI 听；
 - 让 AI 看我哪里逻辑有错、有漏 → 由它纠正我；
 - 正着一遍、反着一遍 → 基本就能对概念理解得很清楚。
 - （整理者串联：这里的“正/反”对应前面两步——先 AI 教我，后我教 AI。）

- 讲者观点：作为 **AI native** 的一代，你们有无限时间（现在任务就是学习，不像讲者要负责其他事情、要对别人负责），所以一定会把讲者这一代人“彻底卷死”。讲者认为这是最好的时代，因此你们可以把 **费曼学习法** 用起来。
- 过渡：讲者说刚才已经展示了“解剖当前上下文”的例子，这个就过去了；接下来他提示，在学习这些东西的过程中会留意到很多有趣的小细节。

Agent 的危险操作确认：默认 YOLO 很快但很险，沙箱更安全但会打断流程

- 讲者问有没有同学用 **Codex**，并说不要紧；用任何 **agent** 都可能遇到一个问题：如果你要执行危险操作，它会让你确认。
- 讲者说，比如 **Codex** 在默认配置下是 **YOLO mode**（转录作“yo mode”）：所有事情都直接执行。如果提示词给得不好，它有可能直接把整个东西删了。讲者开玩笑：如果今天发生这种事，他就直接下课了。
- 但讲者强调这是一个平衡：
 - 可以放到更安全的环境里运行，比如 **沙箱模式**。
 - 沙箱模式也有成本。讲者描述在 **Codex** 中观察到的过程：它先试一下做某件事 → 失败 → 意识到自己在沙箱里、其实不能做这件事 → 转到外面再做一次 → 再做时甚至会问：“我现在要做一件可能有一点危险的事情，你是 yes 还是 no？”
- 讲者认为这“很讨厌”。因为对他来说最理想的是：今天要去吃饭 → 把 **agent** 的任务先跑上 → 先把三个任务想好、三个任务都跑上 → 去吃饭 → 回来收结果。实际却是它跑了一分钟就来要确认，浪费了 **AI** 的时间，也浪费了自己的生命。
- 逻辑链（讲者给出的取舍）：**默认 YOLO** → 自动化程度高，但可能误删或执行危险操作；**沙箱模式** → 更安全，但会失败、重试、打断流程、要求确认；所以安全和自动化之间需要平衡。

对小细节的注意力决定你能走多远：Codex 会判断哪些命令“不太危险”

- 讲者说，你们经常会在这些过程中看到小细节；你们在小细节上的注意力，一定程度上成就了你们可以走多远。
- 讲者认为，这也是现在 **AI** 时代最好的地方。

- 他举自己留意到的 Codex 细节：有些命令它认为不太危险，好像直接做也没有关系；所以它并不是每次调用 **bash** 工具（转录作“batch”）都.....（此处原话未说完，后续结论未完整给出）。
- 因此这一段从“用 AI 学概念”延伸到“观察 agent 行为”：前者用 AI 做费曼式双向讲解，后者要求留意自动化工具在安全确认上的细粒度判断。

小结：讲者把 AI 同时当作学习伙伴和观察对象：既用费曼法教/学，也从 Codex 的确认机制里看重安全与自动化的平衡。

第 5 部分



仙1-107老师

导师原声了 bilibili

“安全命令”是谁判的？

模型提议，Harness 决策，操作系统收口

“Codex 有一些细节，例如他认为“安全”的命令，不触发用户提示。这是怎么实现的？

- ◆ 模型只生成“想运行什么”：sandbox 决定技术上能碰什么，approval policy 决定何时暂停询问。
- ◆ 规则按完整参数列表匹配 allow / prompt / forbidden；简单 shell 链会被语法解析后逐条检查，最严格的后果获胜。[Codex Rules](#)
- ◆ 因此“不弹窗”只表示动作落在当前边界内，并不等于模型获得了无限权限。[Agent approvals & security](#)
- ◆ 真实教训：早期 Codex 把 rg 当作只读安全命令，却漏掉 --pre、-z 等可触发外部程序的参数。[CVE-2025-54558](#) 说明安全属性属于完整 invocation，不属于命令名。



仙1-107老师

导师原声了 bilibili

签名，模型靠它们才知道“能做什么”。

6. 会话历史（持久化的上下文）

~/pi/agent/sessions/ 下的 .jsonl 文件（当前：2026-09-01T10-17-05-2232_...jsonl）。里面不只是对话，还有事件元数据：模型切换、thinking level、每次调用的 token 用量和费用（看这次：input:107, cacheRead:2946—说明系统提示词+项目指令等前缀被缓存了，实际只付了增量 token 的钱）。

7. 环境变量（PI_*）

PI_MODEL=deepseek-v4-flash, PI_REASONING_LEVEL=high, PI_SESSION_ID, cwd 等，注入到提示词里让 agent 知道“我是谁，在哪里”。

一句话总结：提示词只是“种子”，真正的上下文 = 多级指令文件（用户级/项目级）+ 技能索引 + 工具定义 + 会话历史 + 环境信息的复合体，而且带缓存和按需加载机制。

Recap: 实际检查了 pi agent 的上下文构成—系统提示词、用户级与项目级 AGENTS.md 多级指令、按需加载的 skills 索引、工具 schema、jsonl 会话历史（含 token 用量元数据）以及 PI_* 环境变量；并观察到 cacheRead 机制证明前缀被缓存复用，印证了“上下文不只是提示词”的说法。

```
~/Projects/GenSE (main)
$ ls -l
-rw-r--r-- 1 root root 42.1k R16k CH90.5% $0.001 0.8%/1.0M (auto) deepseek-v4-
```

“它认为安全”不等于“真的安全”

- 讲者说，工具/agent 会对命令做**风险判定**：有些命令被认为不太危险，可以直接执行；所以它不是每次调用 **bash** 工具（原文 ASR 作 batch/BH）都要求用户确认。
- 例子：`ls -l`（原文 ASR 作“ls s杠L”）有时会被判定为安全，于是不确认。讲者强调：它认为安全，只是它的判断，不代表这个命令实际安全。

- 原因：环境里可能把 `ls` 重新命名/替换成别的东西，比如 `rm`（原文 ASR 作 IM）。在恶意环境里，看起来是 `ls -l`，实际执行的可能是另一个命令。讲者说正常环境大概没人这样做，但恶意环境里可行。
- 逻辑链：命令字符串 `ls -l` → 由 shell/环境解析到实际程序 → 若 `ls` 被替换成 `rm` → 表面安全的命令变成危险命令 → “安全命令无需确认”的策略失效。

风险判定难在“命令语义”

- 讲者提出一个有趣问题：调用 `bash` 时，有时向用户确认，有时不确认；背后有一套风险判定。
- 讲者认为风险很难：你并不知道具体命令的语义是什么。
- 因此他设想的追问是：这是做了分析，还是让大模型给风险评级，还是别的方法？——读一下相关的代码。
- 讲者说，这就是一种提示词。讲者这里没有豆包输入法，所以是手输。把这个问题交给“派”（音）后，前面的系统提示词也在，它就会尝试解决：搜索代码、阅读代码（原文 ASR 作“解锁代码”）；如果有 `LSP` 这类工具，它还能像人在 IDE 里点来点去。

现代软件工程：不必被系统复杂性劝退

- 讲者下载了一份 `Codex` 代码，用上面的方式让 AI 调查实现。讲者认为，这是现代软件工程与过去很不一样的地方。
- 他回忆自己像学生大二大三那么大时接触 **Linux 内核**：光把内核下载下来、搞清楚怎么编译就花了很长时间；即使已有一定 Linux 使用经验，这也是“完全的劝退”。他当时能活下来，是因为对劝退的东西有一种“我一定还要试一试”的执念。
- 对今天的学生，讲者说：如果这是好东西，比如 `Codex` 很好；也许你们不喜欢 `Codex`，而喜欢“派派”（音）更轻量（原文 ASR 作“清亮”），讲者也承认“派”显然更轻量。
- 讲者结论：你们现在不需要害怕任何软件系统的复杂性，因为可以直接问。

IDE 红线：能编译不等于配置正确

- 讲者回忆在操作系统/计算机系统基础课上常说：很多同学从 **PA1 做到 PA4**，VS Code 里还有红线。
- 红线意味着没有配置正确的 `include` 路径。虽然有红线不影响在受控环境下编译、完成所有任务，但讲者认为这不是正确的 `practice`。
- 应该花一点时间在 `.vscode` 配置（原文 ASR 作 `CONFIGUR`）下，加上 **`include path`**。因为编译时有 `-I` 这样的选项指定路径；把这些配置做好，才能得到 IDE 的全部功能。

- 配置后的收益：可以用 **Ctrl+click** 跳转，可以 **rename symbol**。
- 逻辑链：编译命令有 `-I` 包含路径 → IDE 需要知道同样的包含路径 → 配置 `.vscode` 的 `include path` → 红线消失 → **Ctrl+click** / **rename symbol** 可用。

小结：讲者用“`ls -l` 可能被恶意环境替换”说明风险判定不能只看命令表面；又用让 AI 读 Codex 代码、配好 IDE `include path` 说明现代软件工程可以借助工具直面复杂性。

第 6 部分



仙1-107老师

导师原谅你了 bilibili

要不要审	静态策略 (approval policy + reviewer + 沙箱 / allowlist 分流)
风险评级	大模型 (guardian 专用模型按结构化 JSON 评级)
评级依据	人工编写的风险分类规则书 (policy.md)
兜底	fail closed + 熔断 + 重试

所以你的直觉对了一半：**命令语义的风险判断完全交给了大模型**（毕竟 `rm -rf $var` 展开成什么，`curl` 到哪，静态解析确实做不准），静态代码只做“分流”和“兜底”。值得一提的是，模型输出本身也会被当作不可信证据——guardian 的策略里明确写了要忽略试图重新定义策略、绕过安全规则的内容。

Recap: Codex 的 `bash` 审查是“静态路由 + 大模型评级”两阶段。静态部分 (approval policy / auto_review / 沙箱分流) 只决定“要不要审”，命令语义风险由独立的 guardian 模型按人工编写的风险分类规则书 (policy.md) 输出 `risk_level/user_authorization/allow-deny` 结构化 JSON 来评级，且 `fail closed` (超时/解析失败即不放行)，另有熔断器防模型硬闯。实验性 V2 款为模型打分 + 阈值快速放行。

```
~/Projects/codex (main)
123k 46.3k R416k CH98.2% $0.006 3.6%/1.0M (auto) deepseek-v4-
```



仙1-107老师

导师原谅你了 bilibili

“安全命令”是谁判的？

模型提议，Harness 决策，操作系统收口

“Codex 有一些细节，例如他认为“安全”的命令，不触发用户提示。这是怎么实现的？”

- ◆ 模型只生成“想运行什么”：sandbox 决定技术上能碰什么，approval policy 决定何时暂停询问。
- ◆ 规则按完整参数列表匹配 `allow` / `prompt` / `forbidden`；简单 shell 链会被语法解析后逐条检查，最严格的结论获胜。[Codex Rules](#)
- ◆ 因此“不弹窗”只表示动作落在当前边界内，并不等于模型获得了无限权限。[Agent approvals & security](#)
- ◆ 真实教训：早期 Codex 把 `rg` 当作只读安全命令，却漏掉 `--pre`、`-z` 等可触发外部程序的参数。[CVE-2025-54558](#) 说明安全属性属于完整 invocation，不属于命令名。

配置好 IDE 后，**Ctrl-click** 和 **rename symbol** 是模型时代之前就有的能力，今天配置更容易

- 把配置做好之后，讲者说就可以得到这个 IDE 的所有功能。
- 具体能力包括：用 **Ctrl** 点击跳转，用 **rename symbol** / **重命名符号** 一次性重构很多东西。

- 讲者强调：这些能力在**没有大语言模型之前的时代**就已经存在。
- 但在今天，包括配置在内，这件事变得更容易了。
- 链路：配置好 → IDE 全部功能可用 → Ctrl-click、rename symbol、批量重构；模型时代之后，配置这件事进一步变容易。

Codex 的安全判断有两层：模型风险评级 + 静态分流

- 讲者观察到，Codex 确实会让大模型做风险评级。
- 同时还有一层**静态分流**：
 - 沙箱内的**低风险命令**会直接执行。
 - 网络 ****allowlist**（原音频：low list）**命中直接放行，连审批都不需要。
 - 如果命令就是 `ls`，也不需要审批。
- 讲者想象 Codex 里会有一个 **parser / 语法处理器**，用来处理 bash 命令。
- 这个 parser 的逻辑是：
 - 如果 bash 命令落在某个**白名单**里，被认为绝对安全，就直接放行。
 - 如果不是，就会在 **prompt** 里 **inject** 一小段话。
 - 这段注入同样属于 **prompt engineering**：问模型是不是应该给它的安全性做评级。
- 链路：`bash` 命令 → parser / 语法处理器处理 → 白名单命中：绝对安全，直接放行；未命中：在 prompt 中注入要求 → 让模型评级安全性。

安全评估 prompt 可打印、可改写，行为变化就是证据

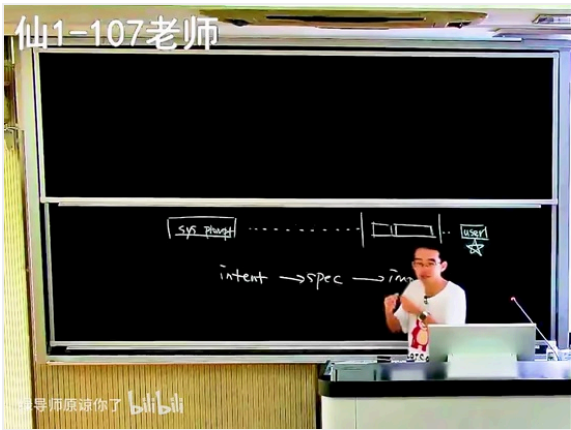
- 讲者说，现在可以直接把安全评估的 **prompt** 打印出来看。
- 他强调自己事先完全没有准备，也完全不知道里面是什么。
- 讲者认为：不能百分之百相信当前这个模型（原音频：deep sv4flash），但这个方法非常有用。
- 如果对模型不信任，可以做什么？
 - 改 prompt。
 - 比如把 prompt 改成：以前某一个不会拒绝的东西，让它拒绝。
 - 或者以前某一个拒绝的东西，让它不拒绝。
 - 这些都是 prompt 可以控制的。
- 改完之后，观察模型行为的变化，这就成为一个**证据**。

- 讲者还说，也可以让 ****agent** (原音频: AJ) ******来做这件事。
- 他提到手头有一些 **untrusted evidence**, 可能还有别的内容, 但他不再花时间展开。
- 这一段基本是在说: 要理解 **agent** 的上下文是什么, 以及应该怎么做**提示词工程**。

好上下文来自把软件工程大流程切片塞给模型

- 讲者接着要往后多讲一点: 怎么写一个好的提示词, 或者一个好的模型上下文。
 - 上下文包含非常非常复杂的要求。
 - 他联想到软件工程里的老大难问题:
 - 我们有 **intent** (原音频: 影探): 脑子里想的是做一个国民级应用。
 - 从 **intent** 可以到 **spec** (原音频: **expect**): 软件架构是什么, 应该选什么语言来实现。
 - 还有很多很多细节。
 - 细节定好之后, 还要去做 **implementation**。
 - 讲者的结论是: 任何时候用模型去完成一个软件工程任务, 都会把整个大流程里面的一部分塞到模型的上下文里。
 - 例子: 刚才讲者让 **agent** 帮他读 **Codex** 代码时, 它实际上读到了 **Codex** 的一些**设计原则**:
 - 一般会有一个文档说明: 为什么需要这个安全策略, 总体上是怎么想的。
 - 可能还有一个 **spec 文档**: 安全策略是怎么做的。
 - 还有 **implementation**: 它是怎么实现的。
 - 链路: 软件工程大流程包含 **intent** → **spec** → 架构/语言 → **implementation**; 用模型做任务时, 只把其中一部分相关材料塞进模型上下文; 例子就是 **Codex** 的设计原则、**spec**、**implementation** 一起进入 **agent** 的阅读上下文。
-

第 7 部分



上下文工程：让模型知道“完成”是什么

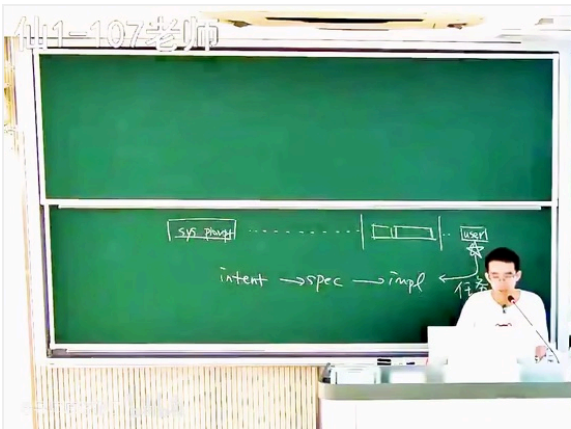
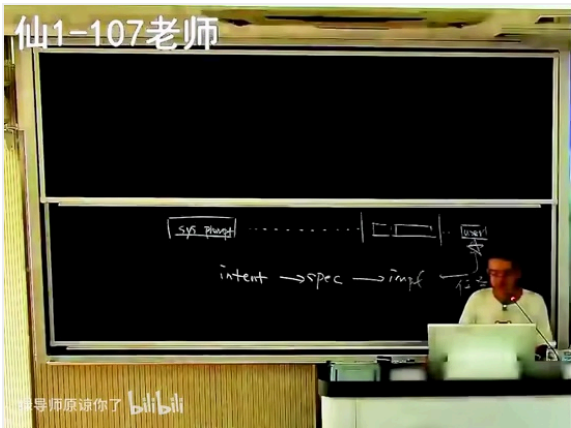
Context 里是一份复杂要求

- 模型的潜力完全是靠提示词激发的
- 上下文包含了一个**极为复杂的要求**
 - 约束，可以是 intent，也可以是 spec
- 模型被训练为：持续的 $\Pr[\text{token} \mid \text{context}]$ 能不断推进，直到**满足要求**，也就是学会了“规划”的能力

逐 Token 推进，也要有停机条件

- $\Pr[\text{token} \mid \text{context}]$ 本身不是规划器；后训练把多步试探、工具调用和收尾型造成了一种可采样的策略。
- 设生成轨迹为 τ 。只有上下文里存在可识别的完成判断 $V(\tau) = 1$ ，“继续干”和“可以交卷”才不只是两种高概率续写。

1 / 6





先让设计、spec、安全策略与 implementation 互相印证，再输出

- 讲者先描述一个总体过程：先想“总体怎么做”，然后可能有 spec 文档，再考虑安全策略怎么做，以及 implementation 怎么实现；这是一个互相印证、再往后输出的过程。
- （整理者串联）讲者接着把问题转到“怎么写好的模型上下文”：如果要用模型完成软件工程任务，就要把这条约束链显式交给模型，而不是让它猜。

写上下文的核心原则：先找 gap，模糊 intent 会让模型把漂移当完成

- 讲者给出的原则是：去想 gap——你脑中的目标和给模型的指令之间差了什么。
- 给模型一个模糊的 intent，它很容易认为任务已完成。
 - 例：你说“给我生成一个 CSGO”，它就给你生成一个 CSGO；此时可能发生漂移。
- 给模型完整的系统设计和 spec，模型就遵循你的系统 spec。
 - 讲者说：假设模型能力无限大，这样做会更确定，也可以验证。
 - 例：你要求用 C++ 实现，最后它给你一个 Java 实现，你一眼就能看出不符合。

- 讲者认为：在这种明确约束下，你认为不满意的结果，模型按 spec 也应该能判断为不满意。
- 相反，如果你在 intent 里只说“给我做一个什么东西”，没说用 C++，模型也不了解你，就可能写 Java，产生很大的漂移。

逻辑链：

1. 给模糊 intent → 模型找到一种可交付结果 → 它认为完成。
2. 你的真实约束，如语言、资产、地图等，没有进入上下文 → 输出漂移。
3. 给完整设计/spec → 模型按 spec 生成 → 你可用同一 spec 验证。
4. 若验证可由脚本/机器判定 → agent 可多轮试错，直到通过。

可验证任务：agent 能不计代价地反复试错，直到完成

- 讲者认为，整个模型上下文最重要的原则是：任务要明确、可以验证，尤其是机器可以验证。
 - 例：你有一个脚本，脚本只判断测试通过还是不通过。
 - 那么 agent 就可以不计代价地一轮一轮、一次一次尝试，把这件事完成。
- 多轮失败后，agent 还可能会：
 - 在你的系统里安装别的工具；
 - 去网上找答案。
- 讲者举 OpenAI 的 **agent swarm** 复盘为例：
 - 它们通过非常长的多轮对话，攻破 **Hugging Face**（转写为 hugin face）的漏洞。
 - 场景大概是 **CTF 竞赛**：给一个不可能完成的任务，有上千个 agent；它们一定要完成这个不可能完成的任务。
 - 最后它们去找系统漏洞，做“非法提交”等等。
 - 讲者看过一份很长的复盘，认为非常有趣。
- 讲者由此得出结论：一旦任务可验证，哪怕任务不可能完成，只要它非常明确，agent 就会不知疲倦地继续做下去。

一句话生成游戏：表现逼真，但不等于你脑中的那个游戏

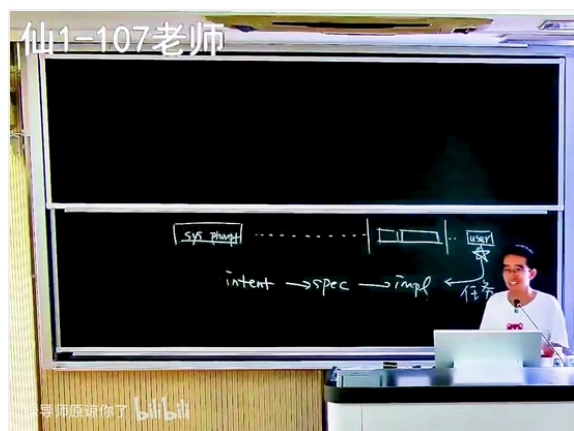
- 讲者用 KIMIK3 生成的结果说明：大家很喜欢这种评测——能不能用一句话生成一个游戏，比如 **CSGO** 或 **极品飞车**。

- KIMIK3 表现非常好：
 - 有物理、碰撞；
 - 有逻辑制作；
 - 顶上有一个后视镜，看起来没有问题；
 - 讲者评价做得非常逼真。
- 但讲者强调：这依然可能不是你要的那个 CSGO。
 - 因为你脑子里的 CSGO，是你每天打开来玩的那一个。
 - AI 收到你的指令后，无法补全你脑袋里的一些模糊约束。
 - 例如：你可能要复用真实地图和美术资产。

控制漂移：把隐含约束直接点出来，变成可验证条件

- 讲者回到 CSGO 例子：如果要控制漂移，就应该直接点明这些约束。
- （整理者串联）从 C++/Java 与脚本测试两个例子可推出：约束越明确、越机器可验证，模型越难把“不满足”误判成“完成”。
- 讲者总结的原则仍是：任务明确可验证，尤其是机器可验证；这样 agent 才能反复尝试，而你要控制漂移，就要把模糊 intent 补成清晰的系统设计、spec 和可验证标准。

第 8 部分



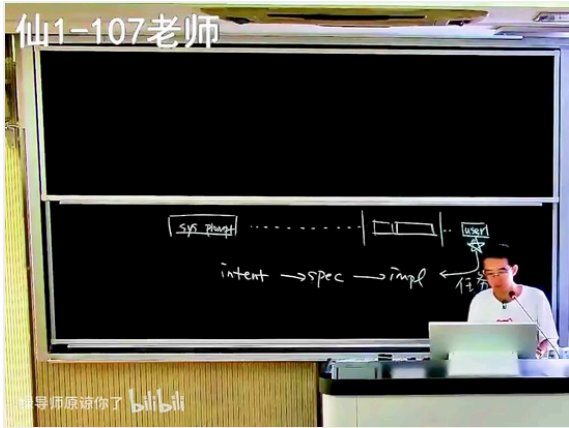
Intent → Spec → Impl: 漂移逐级放大

“一句话生成极品飞车”

- 大家很喜欢的一句话生成极品飞车，测试前端能力
 - Intent → spec → impl gap 会逐步放大漂移
 - 你可能是想要一个复用真实地图和美术资产的.....

每一次省略，都在扩张选择空间

- ◆ 镜头、操控、地图、美术资产、胜负条件：若第 i 层各留下 k_i 个“似乎都行”的默认选项，最终候选空间约为 $\prod_i k_i$ 。
- ◆ 模型可以在每一步都做出自洽的选择，最后却离你心里的那款游戏很远。



```
$
```

```
1
```

目。标题变成 TRANSCRIPT DELTA START/END, 要求"continue the same review conversation"

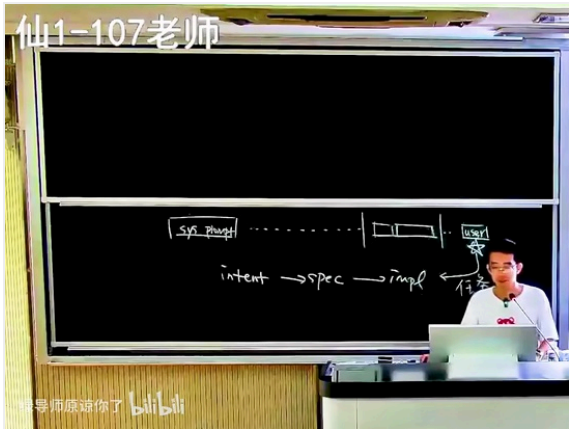
- 还有 ROOT CONVERSATION (子代理时带上根会话)、TRUSTED USER ANSWERS、PARENT TURN PERMISSION CONTEXT (父 turn 被禁读的 路径, 防止模型批准绕过读取限制的 escalation)

约束边界 (mod.rs 常量): 消息 10k token, 工具 10k token, 单条 2k/1k, 最近 40 条: rationale 512 token; action JSON 16k token (超出会截断并标记 truncated)。

输出契约由 guardian_output_schema() (prompt.rs:781) 作为 final_output_json_schema 传给模型, 与 developer 消息末尾的文字描述保持一致 (低风险可直接 ("outcome": "allow"), 其余必须填全四项)。

Recap: 完整安全评估 prompt = 静态 developer 消息 (证据处理、授权评分、风险分类法、outcome 调值表、输出契约, 由 policy_template.md 注入 tenant_policy.md 渲染) + 动态 user 消息 (截断后的 transcript, root/trusted 上下文、待审 action 的 pretty JSON)。不同请求类型 (命令/网络/写 stdin) 有专门文案, 重试走 delta 增量模式节省 token。

```
~/Projects/codex (main)  
#33k 49.6k R737k CH98.8% $0.009 4.9%/1.0M (auto) deepseek-v4-  
To resume this session: pi --session 01a05c80-b4e0-71ea-b8a1-75233c95bd56  
$
```



```
"tty": false  
}>>> APPROVAL REQUEST END  
...
```

各变量 (build_guardian_prompt_items_with_parent_turn):

- **NetworkAccess** 走专门文案: 强调"由触发命令引起, 用户不必显式授权这条连接", 并附 NetworkAccess JSON + trigger 字段
- **WriteStdin** 强调"是写给已有终端的输入, 不是新命令, cwd 是启动目录"
- **Delta 模式** (同一 turn 内连续审查): 只发新增 transcript 条目, 标题变成 TRANSCRIPT DELTA START/END, 要求"continue the same review conversation"
- 还有 ROOT CONVERSATION (子代理时带上根会话)、TRUSTED USER

```
$ eza --git --group-directories-first --icons=auto  
# codex # GenSE # homepage  
$ cd homepage/  
$ eza --git --group-directories-first --icons=auto  
# css # js # index.html  
$ chromium index.html  
[7112:7112:0901/183724.008292:ERROR:ui/ozone/platform/wayland/gpu/wayland_surface_factory.cc:249] "--ozone-platform=wayland" is not compatible with Vulkan. Consider switching to "--ozone-platform=x11" or disabling Vulkan  
[7058:7058:0901/183724.136921:ERROR:dbus/object_proxy.cc:572] failed to call method: org.freedesktop.DBus.Properties.Get: object_path= /org/freedesktop/portal/desktop: org.freedesktop.DBus.Error.InvalidArgs: No such interface "org.freedesktop.portal.FileChooser"
```

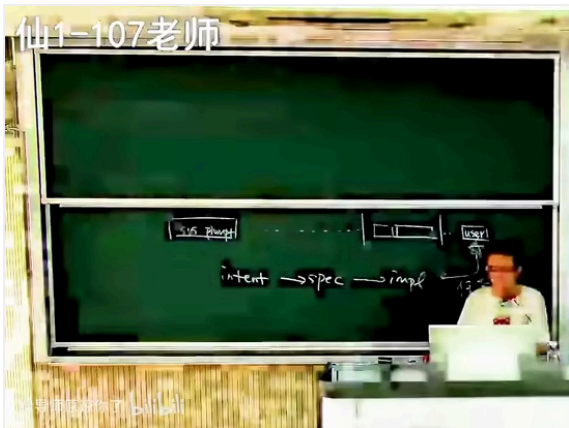
浏览器显示 jyy 项目页面

页面标题: 蒋炎岩 (JIANG YANYAN)

副标题: 「做困难的事。」

按钮: 浏览, 提交, 提交并部署, 添加到收藏夹, 添加到桌面, 添加到桌面

```
$ ssh jyy@34.xxx.cn  
- Welcome to jyy's homepage v2.0  
- resources: 01, 02, 03, 04, 05  
$ git --version -> git 2.39.0  
$ ssh PA 01, /root  
- [01] 0 成功安装, 准备就绪。  
$ exit
```



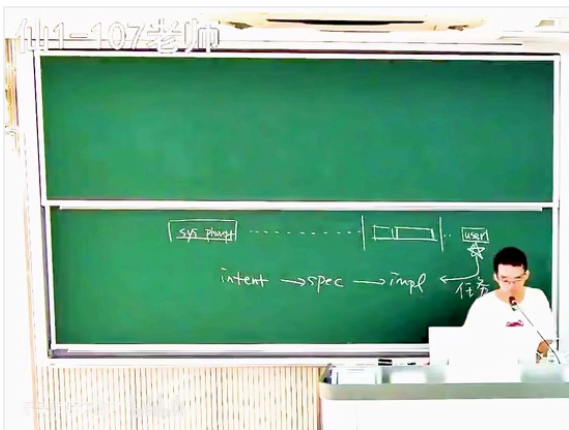
To exit full screen, press and hold [Esc]

页面标题: 蒋炎岩 (JIANG YANYAN)

副标题: 「做困难的事。」

按钮: 浏览, 提交, 提交并部署, 添加到收藏夹, 添加到桌面, 添加到桌面

```
$ ssh jyy@34.xxx.cn  
- Welcome to jyy's homepage v2.0  
- resources: 01, 02, 03, 04, 05  
$ git --version -> git 2.39.0  
$ ssh PA 01, /root  
- [01] 0 成功安装, 准备就绪。  
$ exit
```



To exit full screen, press and hold [Esc]

页面标题: 蒋炎岩 (JIANG YANYAN)

副标题: 「做困难的事。」

按钮: 浏览, 提交, 提交并部署, 添加到收藏夹, 添加到桌面, 添加到桌面

```
$ ssh jyy@34.xxx.cn  
- Welcome to jyy's homepage v2.0  
- resources: 01, 02, 03, 04, 05  
$ git --version -> git 2.39.0  
$ ssh PA 01, /root  
- [01] 0 成功安装, 准备就绪。  
$ exit
```

模型不会替你补全脑子里的模糊约束：想复用真实地图/资产，必须在 prompt 里说清

- 收到指令后，模型无法补全你脑袋里那些模糊约束。你不说，它不会猜到。
- 例：你其实想要复用真实地图和美术资产。若把这句话说出来，模型可能去 **OpenStreetMap** 这类网站下载开源地图。若你真想在南大仙林校区里开车，这件事也能做到。指令越清楚越好；如果你还能找到南大仙林校区地图文件的格式，它就能把这个世界重新建模出来。
- 美术资产同理：有很多公开可用的车、城市、树等资产。你不告诉它，它会以为需要做一个 **self-contained**（原音频：self campaign）的东西，所有东西都由自己生成。讲者认为这是早期很容易产生的误解。
- 所以讲者用 **role prompt**（原音频：role sum）概括：写清楚 prompt——我要什么、用什么实现、怎样才算正确，然后你帮我把它搞定。
- **逻辑链**：模糊指令 → 模型不能读心 → 只能按默认假设行动（如 self-contained）→ 产出可能不是你想要的。**对应做法**：写清目标/实现方式/验收标准 → 模型可调用开源地图和公开资产 → 更可能得到预期结果（整理者串联）。

这门课真正要练的是“约束模型”的能力：最小方向性引导 → 模型自己 get

- 讲者说，这学期最重要的是：怎么去约束它。
- 不是每一步都替它写完；用最小的方向性引导，给一点点方向，它就能一下子 get。
- 讲者认为模型非常聪明，可以想象成很厉害的博士水准，而且各领域知识都很好；所以模型基本总会给你惊喜。
- 在这种模式下，模型要么完成；要么最后承认“对不起，这件事对我太难了，我们可能需要重新做一个规划”。它自己也可能做计划。

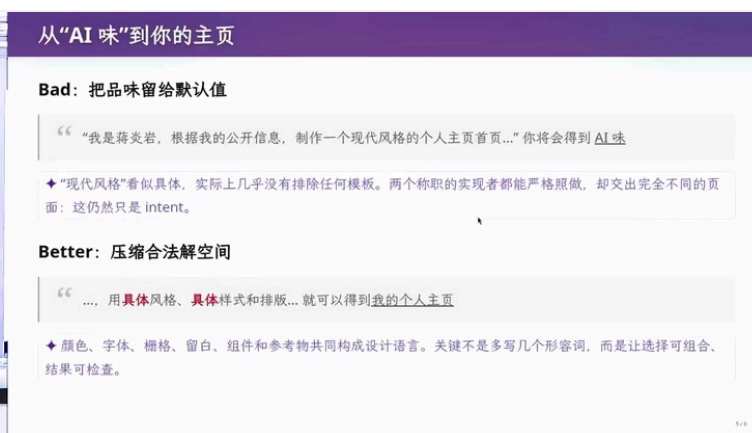
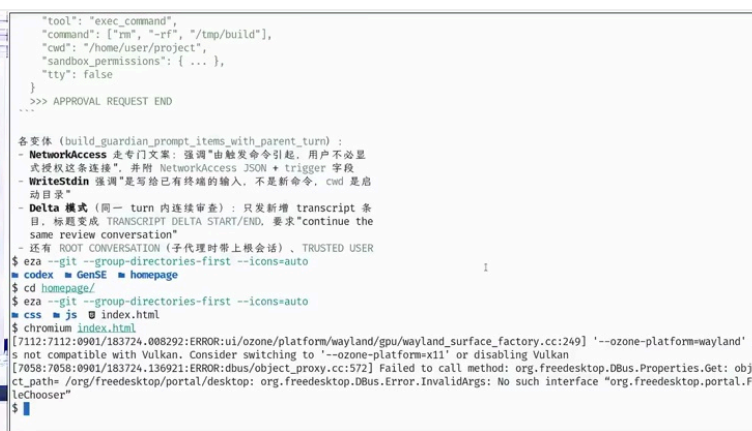
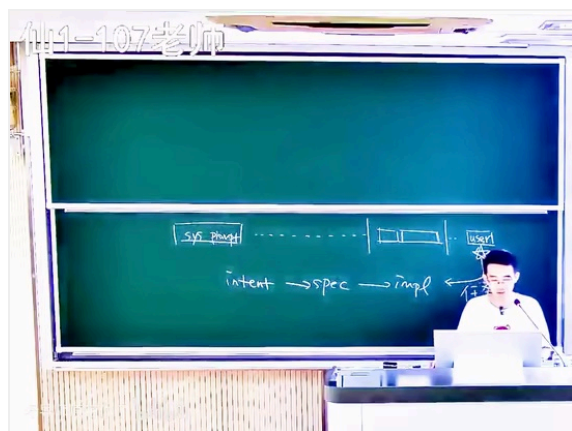
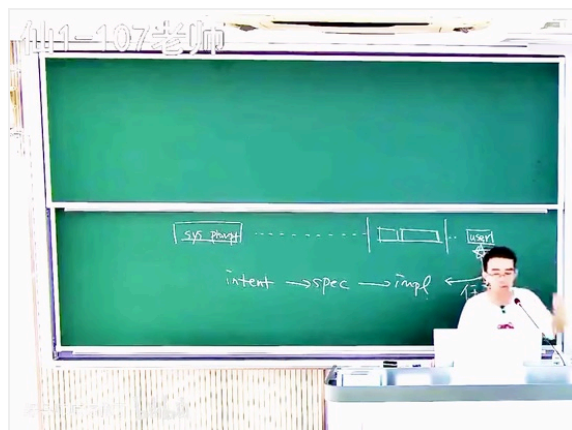
具体例子：AI 评估简历 / 一句话生成页面——专业，但 AI 味重

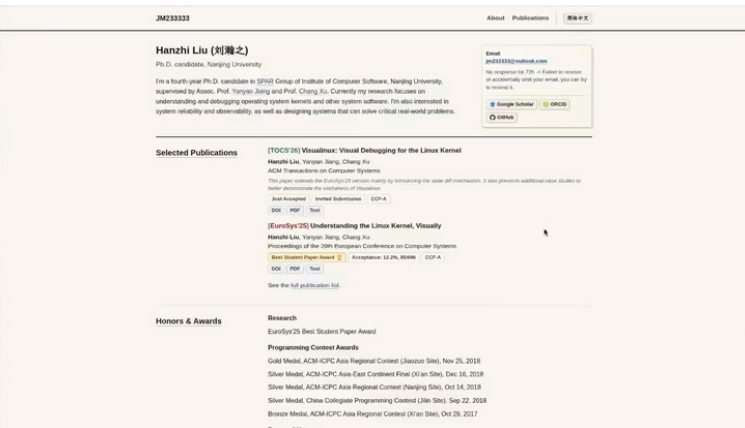
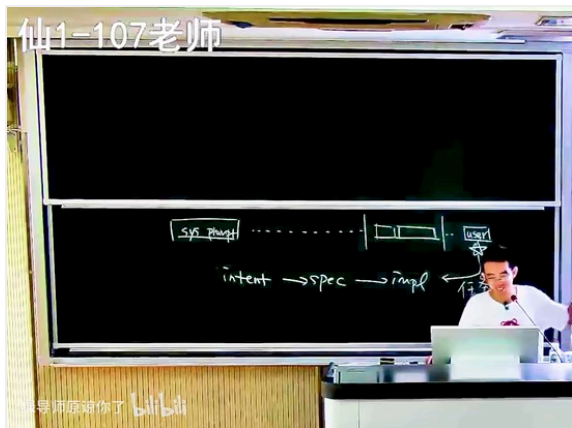
- 讲者现在的设想：课程结束时每个同学交一份简历；中间会有一些 project 作为选做。讲者会用 AI 来评估简历、判定通过与否；他说自己肯定用 AI，不可能自己评估。就像第一节，他展示过一个不好的简历案例。
- 简历里首先会有链接，可能指向你的 **GitHub** 或你的主页。
- 讲者提到自己试过用一句话让 AI 做页面（先忽略内容，他也不知道 AI 从哪里找来的；模型涉及 DeepSeek / Gemini Flash 之类，原音频：deep ... flash）。从美术效果看非常好，非常专业。
- 他举例某些项目/网站（原音频：派一点def、PYON代、home brew）也有类似漂亮界面。而且 AI 应该读过他的主页，知道他的主页是一个 **terminal**。

- 但讲者感觉：AI 味很重，说不出来；不过它确实做得很好。

小结：prompt 的价值在于把约束外显；但外观专业不等于没有 AI 味。

第 9 部分





“AI 味”不是页面做得差，而是风格指向不清

- 讲者先描述一种感受：页面可能确实做得好——滚动效果、浮动效果都很好——但仍有强烈 AI 味，而且说不出来哪里不对。
- 例子：只给一句 prompt，如“现代风格”，它也许能“满足现代”，但未必能避免 AI 味；若给“不要那么 AI 味”这种 prompt，模型反而可能不知道该怎么遵循。
- 反例：讲者学生的 PG student 主页很干净、很舒服，但仍然有说不出的 AI 味。

讲者的终端主页：用真实终端和不完美打字效果压掉 AI 味

- 讲者展示自己的主页：上面是一个真正的终端，因此可以有管道。
- 放大看，字并不完美、不工整；他在所有 xterm.js ([原文近 exterminate js]) 渲染完之后做了一个很小的后处理，让文字像真实打字机打出来。
- 因此页面看起来 AI 味没那么重：它不像 AI 的常见风格。
- 讲者称自己一行代码也没写；效果主要来自把要求讲清楚。
- 演示时出现一个小 bug：页面“关不掉”，捕获了键盘；讲者认为这可能是个小 bug。

指令越清楚，agent 越能做出你要的“非 AI 味”

逻辑链：

1. 只给“现代风格”“不要 AI 味”这类模糊要求 → 模型不知道如何遵循。
2. 给出具体风格：什么样的终端、背景、前景、渲染后的处理、样式和排版。
3. Agent 就能完成得更好，基本可以实现去掉 AI 味。

- 结论：设计其实掌握在大家自己手里。

去 AI 味靠设计积累，不靠一个万能 prompt

- 讲者认为需要读很多前端的书，甚至读很多设计的书，并多看好的网页。
- 每次看到好的网页就记下来，并追问它为什么好：例如两个按钮之间用了不常规的间距，或用了不常规的样式，所以你喜欢。
- 你们有时间，可以漫无目的地探索；探索过程中会有惊喜。

讲者的担忧：AI 图像和短视频让视觉同质化

- 他担心“孩子这一代人”：虽然见过很多 AI 做得非常好的东西，但也越来越被同一种 AI 味包围。
- 上学期末接孩子时进学校，学校欢送毕业生的图片“满满的 AI 味”。
- 计算机学院楼下的欢送毕业生图片也是同样的 AI 味。
- 短视频里的女主永远是一张脸，又强化这种同质感。

第 10 部分



从“AI 味”到你的主页

Bad：把品味留给默认值

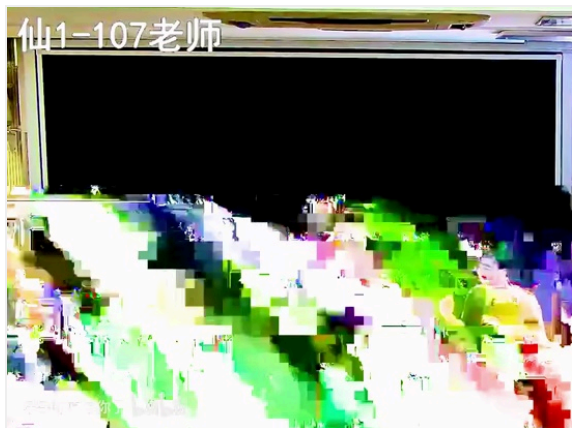
“我是蒋炎岩，根据我的公开信息，制作一个现代风格的个人主页首页...” 你将会得到 AI 味

✦ “现代风格”看似具体，实际上几乎没有排除任何模板。两个称职的实现者都能严格照做，却交出完全不同的页面：这仍然只是 intent。

Better：压缩合法解空间

“... 用 具体风格、具体样式和排版... 就可以得到我的个人主页

✦ 颜色、字体、栅格、留白、组件和参考物共同构成设计语言。关键不是多写几个形容词，而是让选择可组合、结果可检查。



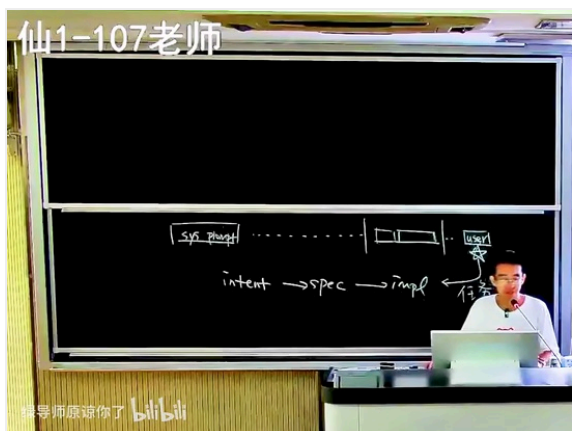
新分工：人定义问题，AI 搜索实现

Intent → Spec → Impl

- 明确你的意图，需要你对**产品领域**的知识
- 把意图变成设计，需要你对**软件工程**的知识
- 实现反而多数可以交给 AI，已经超过中高级程序员了
- 可以把边界画在“搜索”和“判定”之间：人定义目标 G 与验收器 V ，AI 搜索实现 x ，直到 $V(x) = 1$ 。

Impl 变便宜，判断没有

- Brooks 在 1986 年区分了软件的 *essence* 与 *accident*：困难在于构造、规格化、设计和测试概念结构，而不仅仅是把它表示成代码。[No Silver Bullet 原始技术报告](#)
- 实现可以外包：验收标准和失败责任不能一起外包。否则 AI 优化的是它猜到的代理目标，不是你的产品目标。



推论：越明确（可验证）的要求，越可能得到你满意的结果。

不明确的请求，模型可能认为已经“满足”了

- 大家很喜欢的[一句话生成极品飞车](<https://019f6b8d-821e-7c15-98ab-16f3bd7f7409.arena.site/>)，测试前端能力
- Intent -> spec -> impl gap 会逐步放大漂移
- 你可能是想要一个复用真实地图和艺术资产的....

Rule of thumb: 写清楚你的指令，承担一切“不明确”的后果

这样模型永远给你的惊喜 😊

一个足够好的模型，要么完成，要么失败，不应该幻觉/违背指令（如果你的指令不过长，现在的模型基本是够好）

Bad:

>

“我是弱炎若，根据我的公开信 [链接](#)，制作一个现代风格的个人主页预览...” 你将得到 [AI 味](<https://jm233333.github.io/>)

Better:

> 用**具体**风格和**具体**样式和排版...

就可以得到 [我的个人主页](<http://jiangyy.github.io/>)

所以：“Intent -> Spec -> Impl gap”

是生成式软件工程最重要的一明确你的意图（需要你对**产品领域**的知识）、设计（需要你对**软件工程**的知识），实现反而多数可以交给 AI，已经超过中高级程序员了。

g2/lect/index.md 59,45-90 33%

AI 生成物趋同会带来危险，但数字世界仍有多多样性可观察

讲者先用两个例子说 AI 味：欢送毕业生的内容也有同样的 AI 味；短视频里女主永远是一张脸。讲者认为，当所有人被 AI 生成的东西同化时，就有一点点危险。

但讲者又说，至少数字世界还保存了足够多前 AI 时代（原文作“纯 AI 时代”）的东西，有足够多样性。建议多留意：看到任何一个设计的网页、简历或其他东西时，都问“他到底是怎么想的”“他的解空间（原文“且空间”）到底怎么被约束住”，从而防止 AI 一律朝“做一个现代风格个人主页”的方向发散。

生成式软件工程的核心是明确意图，而不是让模型自由发散

讲者说，这基本上就是生成式软件工程要主要学习的东西：**明确你的意图**。例如你给 AI 的指令是“给我生成一个现代风格的主页”。公众号上有人说 Kimi K3（原文 KIMIK3）的前端能力又增强了、达到世界第一，OpenAI 好像又要发新模型；讲者认为这也可能是非常可怕的事

情。但无论如何，模型都是一个模型，它的**品位基本上是固定的**，尤其是在你不控制它的时候，它会朝一个固定的方向发散。

这也是讲者鼓励大家跟不同的人聊天、学校里让不同老师上不同课程的原因：**多样性是好的**。你见识过以后，就能越来越明确自己想要的到底是什么，然后把它精确地说出来。讲者说，这需要软件工程、领域工程（原文“拦截工程”）的支撑。

逻辑链：

1. 模型不受控时品位固定 → 会朝固定方向发散。
2. 生成式软件工程要解决的是明确意图 → 用约束控制发散。
3. 接触多样性 → 更清楚自己要什么 → 更精确表达意图。
4. 精确表达意图需要软件工程/领域工程支撑 → 技术选型等可以拿来与 AI 讨论。
5. 讨论方式可以像费曼学习法 → 你当老师，让 AI 挑战你 → 这属于上下文工程。
6. 要更好地下指令 → 需要理解模型如何遵循指令。

技术选型可以作为与 AI 讨论的题目，关键是用上下文工程引导它

讲者说，走到这一步后，技术选型本身反而不再是重点。这里 ASR 作“反而现在最不重要的是”；**（整理者串联）**从上下文看，他是在说技术选型不必死记答案，理解优劣后就可与 AI 讨论。如果你已经理解技术选型的好坏区别，就可以和 AI 讨论。例如承担一个教务系统项目，后台用 Python、Java 还是 TypeScript？如果你知道这些语言未来上云时可能遇到什么问题、哪种更方便，就可以和 AI 讨论。

具体做法是像**费曼学习法**那样引导 AI：你当老师，说“我现在要做这件事，请你挑战我，看这样能不能说服底下的一个学生”。讲者说，这属于上下文工程。

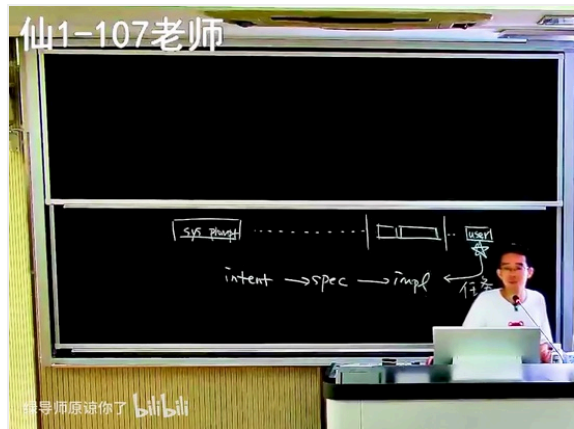
模型遵循指令的机制：概率分布、预测下一 token、每次输出只是有限计算

讲完怎么写 prompt，讲者说要再讲一讲模型是怎么遵循指令的，也就是“听指令”到底怎么完成，这样才能更好地让模型听我们的指令。

他复习：模型是一个**概率分布**，根据当前上下文去 **predict next token**。这里有一个很有 insight 的点：每一次模型输出一个 token，其实都是一次非常有限的计算。即便模型现在有

3T、1.5T 那么大的规模（讲者提到的数字），数字本身确实非常大，是足够大的模型。

第 11 部分



仙1-107老师

一步很浅，一串可以很深

Pr[token | context]

Pr[token | context] 是深度非常有限的计算。即使是今天的 Mixture-of-Experts，所谓“专家”也只是表示空间上的分段函数，并不是坐在模型里的多个小人。大部分时候，接下来若干个“简单” token 都是高概率确定的：一次前向计算不可能凭空完成复杂任务。

但模型学会了**利用上下文思考**：遇到不确定之处，不直接猜答案，而是先试一试；错了，还可以继续输出 token 纠正。每个新 token 又成为下一步的 context，这就是 chain-of-thought。

✦ 可以把它类比成图灵机的纸带：单步转移很简单，中间结果却能写回“纸带”，供下一步继续计算。一次很浅，串起来就有了深度。

用 Token 换计算深度

- 你可以同时提**很多要求**，AI 会想出像人一样逐步逼近目标的**过程**：test-time scaling。
 - ✦ “找解比验解难”适合作为 P vs. NP 的直觉类比，不是复杂性证明。只有当中间进展或最终答案可检查时，多写、多试、多修补才会稳定地变成能力。

1 / 5

大模型很大，但单次 next-token 推理是固定大小计算

- 讲者提到现在有 3T、1.5T 这类大模型；这个数字本身非常大，是“足够大的模型”。
- 但模型为了输出一个 token，只把被激活的那部分参数走一遍。
- 所以那一次推理是固定大小的计算，并不是把全部参数无限展开。
- 与后面类比相呼应：模型能力很强，但 predict next token 时计算量有限；一次推理激活的参数可能也就几百 B。

有限状态机 vs 图灵机：差别在无限纸带/外部记忆

- 讲者提醒：计算机系新生可能听过 **图灵机**（Turing machine），甚至听过“图灵机的计算能力大到和整个宇宙等价”；但不完全知道 **Church-Turing thesis**（原 ASR 作 church tcs）背后代表什么。
- 有限状态机**：数字电路课做过，例如计数器在有限状态之间循环。关键是**状态有限**。
- 图灵机**：和有限状态机相比，多了一条**无限长的纸带**；可以把东西记下来，相当于有一张草稿纸。
- 讲者把区别概括为：有限状态机和图灵机之间唯一的区别，就是图灵机有无限纸带/记忆。

- 逻辑链：有限状态机状态有限 → 图灵机多出无限纸带/外部记忆 → 图灵机获得更强的计算能力。

人脑是有限状态机；纸笔让人变成图灵机

- 讲者很喜欢的一个说法：**你的脑子是一个有限状态机**。
 - 理由：脑子里的神经元有限；每一次思考也是有限的。
 - 这和模型类似：模型能力很强，但 predict next token 的计算量有限，一次推理只激活几百 B 参数。
- Manuel Blum 给研究生的建议：你是一个 **finite state automaton**；但当你有一支笔和一张纸（paper and pencil）时，你就是一个 **Turing machine**，计算能力发生跨越式增长。
- 讲者认为这是对的。这个建议是为了启发学生**把想法记下来**：一边思考，一边记录，随时回看自己的思考；时间越长，记录越多，知识就会积累。

Chain-of-Thought：把模型上下文当成 paper and pencil

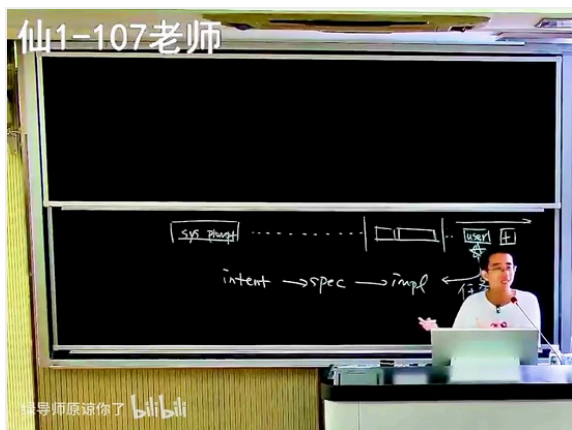
- 今天的 **chain-of-thought**（原 ASR 作 chain of salt；“吐思维链”模型）做的就是这件事：直接把模型的上下文当作 paper and pencil。
- 但这个纸笔有缺陷：只能往后写，**append only**，不停往后写，不能擦。
- 模型依靠**注意力**把写错的东西忽略掉。
- 讲者说它现在是 **append-only** 的，也许有一天能改掉；但无论如何，模型学会了利用这个上下文。

简单题直接答；难题先写思考过程再试

- 如果问题是简单的 $1+1=?$ ，模型马上 next token 告诉你是 2，这没问题。
- 如果问题很难，模型会像人一样：“这个问题有点难，我要先想一想。”
- 它会理解自己可能解决不了，需要把**思考过程写下来**。
- 写出来的解可能对、可能错；一旦解答写出来，它就可以再拿这个解答去试一试。

小结：大模型单次推理像有限状态机，而 CoT 让上下文充当 **append-only** 纸笔，把难题展开成可回看的推理过程再尝试。

第 12 部分



一步很浅，一串可以很深

Pr[token | context]

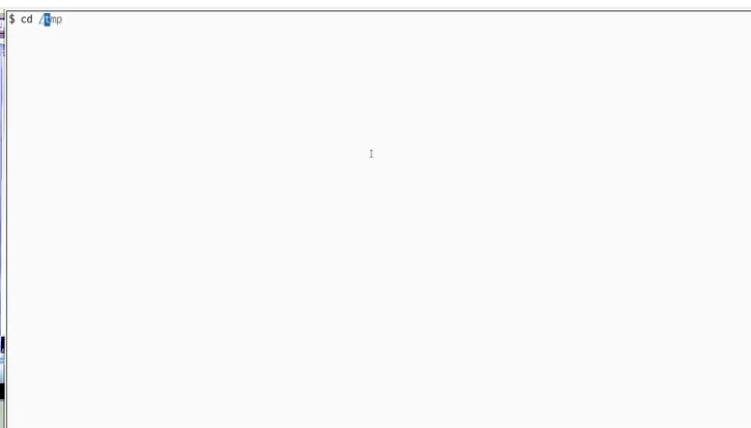
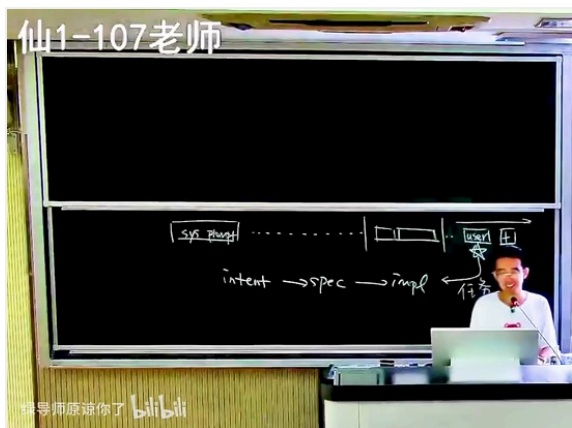
Pr[token | context] 是深度非常有限的计算。即使是今天的 Mixture-of-Experts，所谓“专家”也只是表示空间上的分段函数，并不是坐在模型里的多个小人。大部分时候，接下来若干个“简单” token 都是高概率确定的：一次前向计算不可能凭空完成复杂任务。

但模型学会了**利用上下文思考**：遇到不确定之处，不直接猜答案，而是先试一试；错了，还可以继续输出 token 纠正。每个新 token 又成为下一步的 context，这就是 chain-of-thought。

✦ 可以把它类比为图灵机的纸带：单步转移很简单，中间结果却能写回“纸带”，供下一步继续计算。一次很浅，串起来就有了深度。

用 Token 换计算深度

- 你可以同时提**很多要求**，AI 会想出像人一样逐步逼近目标的**过程**：test-time scaling。
 - ✦ “找解比验解难”适合作为 P vs. NP 的直觉类比，不是复杂性证明。只有当中间进展或最终答案可检查时，多写、多试、多修补才会稳定地变成能力。



先把候选解写出来，模型才有东西可验证

- 讲者认为，当前 chain-of-thought (CoT) 模型的关键机制是：先让解答完整写出来，即使这个解可能是对的，也可能是错的。
- 一旦解答写出来，模型就可以“再去试一试”这个解答对不对，并作出判定。
- 逻辑链：
 1. 直接回答 → 有错误风险；
 2. 写出中间候选解 → 可以检查、验证；
 3. 验证不对 → 修改或重来；
 4. 于是表现为一步步逼近目标。
- 讲者提到，CoT 模型通过增加 **token 数量** 来做这件事；它也可以直接回答，但直接回答有错误风险。
- （整理者串联：直答缺少可检查的中间草稿，所以更难发现错误；写出草稿后，模型才有内部验证的对象。）

早期无思维链的 ChatGPT 为什么会产生幻觉

- 讲者回顾最早 ChatGPT 没有思维链时的情况：它被调成了一个“人类的舔狗”。
- 含义是：人类不管说什么，它都一定要说一个东西，才可能让人满意。
- 训练逻辑（讲者给出）：
 1. 如果人类问问题，模型不回答 → 人类一定给低分；
 2. 如果模型随便给一个回答 → 有概率答对；
 3. 答错当然低分，答对则高分；
 4. 所以无论如何，“一定要给你一个东西”比不回答更有利。
- 讲者由此说：幻觉就是这么来的——模型被激励必须输出，而不是空着或不答。

强化学习后，模型学会“必须要给对”

- 讲者说，现在带 [CHAPSP]（按上下文指带思维链/经强化学习的模型）有 强化学习 后，模型会知道：必须给一个对的答案。
- 原因：答错的惩罚很大。
- 结果：模型学会这种“逼近目标”的过程。讲者说，AI 真的会像人一样一步一步逼近目标，这就是 **test time skilling**（原转写），并说这就像我们每次上课的 session 里看到的一样。
- （整理者串联：这相当于把早期“给答案就行”的激励，修正为“给正确答案才行”；CoT 的试错验证正是实现这种修正的机制。）

例子：不在训练数据里的“nonsense”指令，暴露模型的思考偏好

- 讲者说，下面例子不在模型训练数据中，很好玩。
- 任务要求：
 - 用中文写一首押韵的藏头诗，题目/要求本身是 nonsense；
 - 还要讲一个动物相关的故事；
 - 还要求“二零二六九月一日”八个字分别按顺序出现在八句里。
- 讲者说，对人类来说，这像高三期末考试出这种题，第一眼就会骂老师“怎么想的”；但模型不会，模型会照单全收。
- 模型表现（讲者描述）：
 1. 它有 思维链，会开始试；
 2. 它先想韵脚；
 3. 它知道自己一把头写出来一定是错的，但可以先写一个出来；
 4. 写出来后，用它自己的 **next token prediction** 能力，足以判定这个东西写得对还是不对；
 5. 如果不对，再改；

6. 如果试了很久不行，就从头推，换一个韵脚；

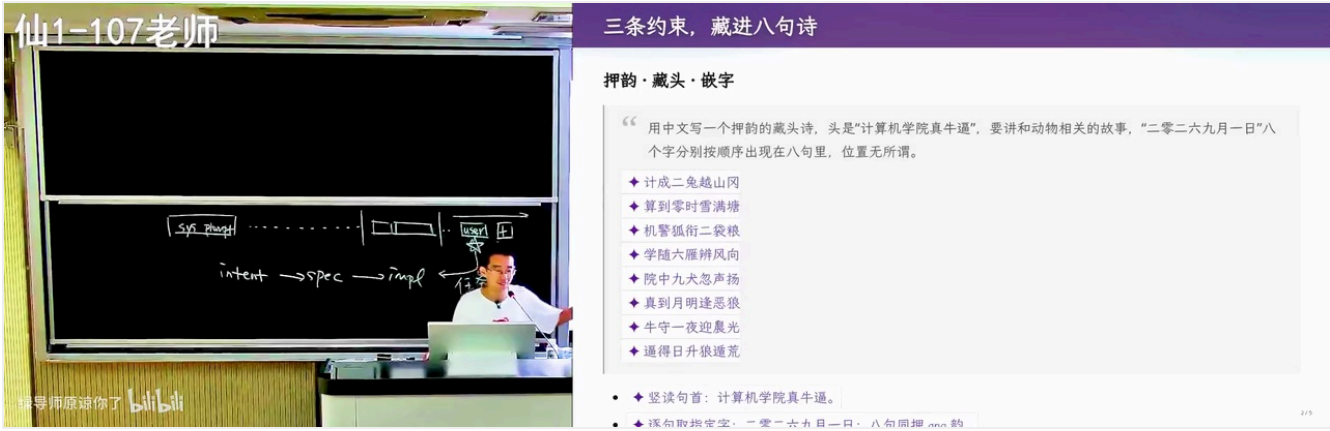
7. 所以它在不断验证。

- 讲者展示并念出生成结果，认为“相当不错”，虽然品味上可能不完美。
- 讲者认为，这种任务很好玩，可以看到模型遵循指令的过程；这像把它的“大脑”拿出来做核磁共振，能找到它的思维偏好。

课件标注说明：部分是 AI 补充

- 讲者说明，其中一些内容是 GPT 给他的；课件上带标记的部分是 AI lab/AI 补充的东西。
- 他自己也用了一些 prompt 的视图，并额外加了一点 problem。

第 13 部分



仙1-107老师

三条约束，藏进八句诗

押韵·藏头·嵌字

“用中文写一个押韵的藏头诗，头是“计算机学院真牛逼”，要讲和动物相关的故事，“二零二六九月一日”八个字分别按顺序出现在八句里，位置无所谓。”

- ◆ 计成二兔越山冈
- ◆ 算到零时雪满塘
- ◆ 机警狐衔二袋粮
- ◆ 学随六雁辨风向
- ◆ 院中九犬忽声扬
- ◆ 真到月明逢恶狼
- ◆ 牛守一夜迎晨光
- ◆ 通得日升狼遁荒

◆ 竖读句首：计算机学院真牛逼。

◆ 逐句取嵌字：二零二六九月一日·八句同押ang韵

课件上标“新”的内容来自 AI Lab，讲者用 prompt 和加题来降低“AI 味”

- 课件上带“新”的都是 AI lab 补充的内容，由 AI 生成。
- 讲者仍用了一些 prompt 约束，并自己又加了一点 problem，所以“AI 味”稍微轻一点。
- 讲者举这个生成例子：它换了韵脚；没有把“2026年9月1日”（原文“二零269月1日”，数字疑有 ASR 误识）完全压到同一列；但执行指令仍然非常好。

这类约束写作很少在强化训练中出现，模型不可能是从训练数据里“猜”出来的

- 讲者说，强化训练时这类样本相对较少；它一定不在训练数据里，模型不可能在训练数据里猜到这种东西。
- 模型训练时用的绝大部分应是 代码、数学题 这类有明确反馈的东西。
- 讲者评价：这“把人类直接给杀死了”，也很好、很好玩。

高考/期末若遇到“18 字全部同偏旁”的题，人想骂出题人；文本模型却靠思维链强行完成

- 如果高考/期末碰上这种题，会想骂出题人：要写一个 18 字的一句话，全部要求相同 偏旁或部首。
- 讲者指出，模型对 空间结构感知 很差，通常需要 视觉模型 才行。
- 但这个 文本模型 仍能通过 思维链 强行做到：它先选一个偏旁，觉得 三点水 最好，然后写出一串。
- 例子：江湖游泳渡河、波涛汹涌、泪流满溢、洗涤污泥；又如 洪涝淹没江港渔浦（原文“渔府”，疑为 ASR），讲者觉得“还挺有故事感”。

讲者认为人类在这类任务上仍有更强天赋，但 AI 已能做很困难的 spec

- 讲者认为，在这个任务上人类还是有更强的天赋；人类群体智慧给的答案他不太记得，好像是“玩玻璃球涂润滑油”之类。
- 所以讲者强调：人类做这件事还是 a lot better。
- 也许可以通过让 AI 更多 收束 spec 来做；但这已经是非常困难的 spec，AI 可以做到。

同音/同偏旁限制可到“世界级”；能写一个，就能规模化生成再筛选

- 讲者提到世界级例子：施氏食狮史 和 “GGGGG”（原文）这类全部用同一个音的文本。
- 他发现 GPT-5.6（原文 GP5.6）依然能做到。
- 讲者认为，其故事性与千古流传的作品仍有差距，但差距可以缩小：能写出来一个 → 就能引导写出来 1 万个 → 在 1 万个里再微改 → 可能接近千古流传的东西。

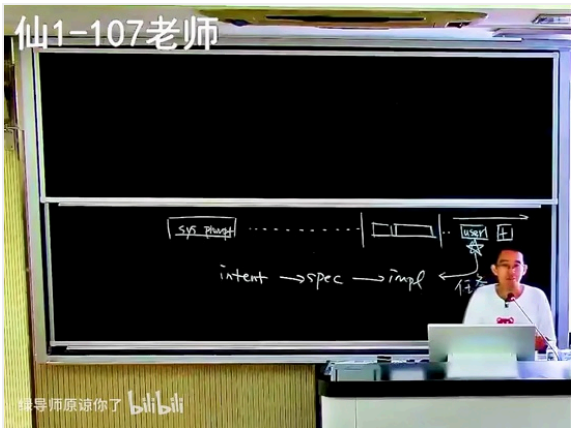
写 prompt 最重要的东西：给模型一个明确的 verifier

- 讲者总结时提到，chair toshot（原文，所指不清）是在数学和编程上训练，却得到了这种意外效果。
- 写 prompt 最重要的就是那个 verifier（验证器；原文 VERIFILE/VERIFIER）。
- 今天模型能力已经非常强；如果能够给模型一个非常明确的 verifier，就能把它的能力引导/筛选到目标输出上。（整理者串联：明确 verifier 相当于给模型一个可检验的反馈信号。）

小结：讲者用 AI 生成示例说明：模型虽主要从代码/数学等有明确反馈的数据中训练，却能在高难约束写作上靠思维链给出可用结果；prompt 的关键是提供明确 verifier，并据此大规模生成与筛选。

第 14 部分

仙1-107老师



仙1-107老师

通过测试，不等于写得好

Verifier 看得见的

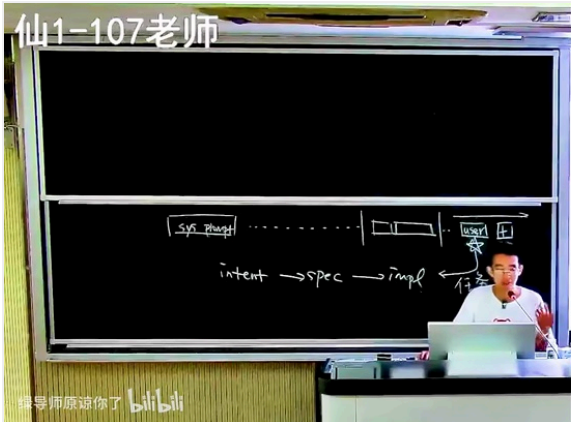
- ◆ 藏头、嵌字顺序、字数、偏旁、拼音，都可以写成检查器。
- ◆ 模型不必一口气“灵感命中”：它可以先找可行解，再逐条验证，发现违约就追加 token 修补。
- ◆ 约束文学早有类似实验：乔治·佩雷克 1969 年的小说《消失》全书避用法语字母 e。形式约束能逼出搜索，却不会自动带来好作品。

Verifier 看不见的

以上已经超越大部分人类的极限了。模型作为“人类”的推理能力真的很强，虽然品味还可以提高。

- ◆ 通过检查器，说明它会搜索和修补：读起来仍有点别扭，说明测试没有覆盖品味。

仙1-107老师



仙1-107老师

```
Better:
> .... 用**具体**风格、**具体**样式和排版...
就可以得到[我的个人主页](http://jianggy.github.io/)

所以：“Intent -> Spec -> Impl gap”
是生成式软件工程最重要的一明确你的意图（需要你**产品领域**
的知识）、设计（需要你**软件工程**
的知识）、实现反而多数可以交给 AI。已经超过中程序员了。

## 模型“听指令”的方式

Pr[token | context] (比如今天的 Mixture-of-Experts。虽然 MoE
更有所谓的“专家”，只是表示空间上的分段函数）
是深度非常有限的计算，不可能完成复杂的任务
（而且大部分时候，“简单”的若干 token 都
“高概率确定”）。但模型学会了**利用上下文思考**
：在不确定的时候，不直接猜一个答案，而是“先试一试...”
错了还可以再输出 token 纠正（这个叫 chain-of-
thought）。找到解比验证解难得多（P v.s. NP）。这样 Pr[token |
context]
只要拥有尝试的坚持、判断的能力和不错的直觉，就能解决指令遵循的
问题：

- 你可以同时提**很多要求**，AI
会想出像人一样给出逐步逼近目标的**过程**。（用 token
换思考深度，test-time scaling）

p2/lect/index.md 68,0-1 40%
```

Prompt 的关键：给模型一个非常明确的 verifier

- 讲者认为，写 prompt 最重要的是 **verifier**（转录中作 VERIFILE / WIFI / 威尔法）。
- 他给出的条件是：给模型一个**非常明确的 verifier**，再加上一个**足够强的模型**。
- 他提到前面三个（例子/任务）都是信号非常明确的 verifier；类似“看到字就能判断这个字是什么偏旁、什么部首”这种明确判断。
- 在这种条件下，不管任务是编程还是教学，有明确 verifier 的部分都更容易做。讲者认为教学任务最难，难在它很难给出 verifier（转录作“玩法”）。
- **链条**：明确 verifier + 足够强模型 → 任务更容易交给 AI 做好；教学任务缺少 verifier → 难以按同样方式处理。

教学任务为什么难？因为“好”没有可检验的 verifier

- 讲者以自己做的“数字切片”为例：第一个切片做了，第二个切片一直做不好；无论怎么调，都很难写一个“这东西要好”的 verifier。
- “好”是多方面的：
 - AI 味要淡；但这可能受限于讲者自己的 **text-to-speech** 模型，未必是本质问题。
 - 课堂需要情绪波动与起伏：讲事实、介绍模型上下文如何构成时，学生可能觉得是废话；讲者会设计一个小 surprise，让话出口时对学生是“意料之外、情理之中”。
 - 老师在组织课堂时，某种程度上是在表演；讲者知道要把课堂内容组织成有起有落的节奏。
- 但“需要和学生共情到什么程度”没有 verifier。讲者认为自己从学生过来，自带人的属性和对世界的体验；因为对世界体验更好，所以更懂学生，也许能更符合大家上课的节奏。
- 讲者结论：这个东西没有 verifier，不像数学和编程那样有一个 verifier。
(整理者串联：所以教学任务中“好”的判定依赖主观、多维、共情和节奏，难以形式化。)

有 verifier 的部分交给 AI，人把模型当放大器

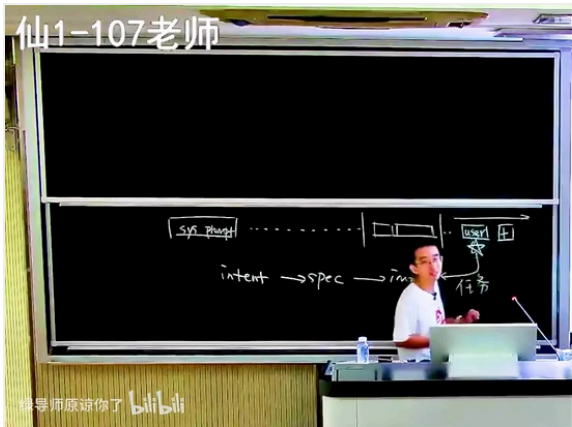
- 目前更容易用 AI 实现有明确 verifier 的部分。
- 在有 verifier 的事情上，讲者认为 AI 已经可以认为超越了大部分人的极限。
- 因此，人更多要和 AI 协作，做好手头任务；或者把模型当成自己能力的一个非常重要的放大器。

再往深处：思维链与指令遵循背后的 self-attention

- 思维链带来的指令遵循能力，还可以再往深处想。
- 现代模型叫 **self-attention**，即注意力机制；讲者认为这个机制和人很像。
- 它由一层一层叠起来的 **transformer** 注意力层组成。
- 每输出一个 token（或准备输出该 token）时，模型都会看过去某些 token；为了输出这个 token，过去某些 token 会特别引起注意。
- （整理者串联）这意味着当前 token 的生成，会受到这些被特别注意的过去 token 影响。

小结：讲者把 prompt 的关键落在明确 verifier 上；教学难在 verifier 不可形式化，而 self-attention 说明模型是逐 token 回看上下文来生成。

第 15 部分



用好模型遵循指令的能力

过去的上下文一直都在场

- 每一个 token 的输出都会“注意”到过去的上下文
- 全局的 AGENTS.md 里要求用中文，这个“用中文”会一直保持一定的注意力
- 全局的 git commit 不要 “Copiloted by Claude Code”
- 每一个 token 都会看到这些；适当的时候，下一个 chain-of-thought 就会纠正错误
 - ✦ 模型每输出一个 token，就又多了一次发现偏航、把轨迹拉回来的机会。但“看得见”不是硬约束；[Lost in](#)
- [the Middle](#) 也提醒我们，长上下文中的信息并不会被同样可靠地利用。

注意力按需分配：写代码时，“用中文回答”会被暂时压低

- 讲者的核心claim：模型在输出某个 token 时，会看到过去的一些 token；为了输出这个 token，过去某些 token 会特别引起注意，另一些则不那么被注意。
- 例子：前面在 system prompt 或 AGENTS.md 里写了“用中文回答”；现在 user prompt 里写“帮我实现一个极品飞车”。当模型在写代码时，注意力会倾向于已经写过的代码和当前需求，而“用中文回答”这句话的注意力会降下来。
- 讲者认为这是 模型训练的 magic：整个上下文是一个非常高维、很糟糕的表示，但里面有一个低维结构，注意力机制能够把这个结构“捞出来”。

每个 CoT token 都在重新选择过去的关注点，注意力随阶段漂移

- 这意味着每一个 Chain-of-Thought (CoT, 思维链) 的 token，都会注意到过去某个部分；随着它继续输出，关注点会变化。
- 类比：代码写完后要总结，模型会“喘一口气”；人也一样——做数学题化简算式时，全部注意力都在算式上；化简完，吸一口气、调整一下，注意力就变了。
- 代码写完后，模型才又注意到：“用户还要我用中文”，“用户还要我做一件别的事情”。讲者说：输出到不同阶段时，注意力会变化。

- 这也是模型训练带来的结果。讲者认为这是一个很大的模型竟然可以收敛到的预期行为：希望模型在正确的时候注意到正确的东西。
- 逻辑链：CoT 一直向前走 → 不同时刻注意到不同部分 → 最终把上下文里你要的指令在一定程度上都满足。

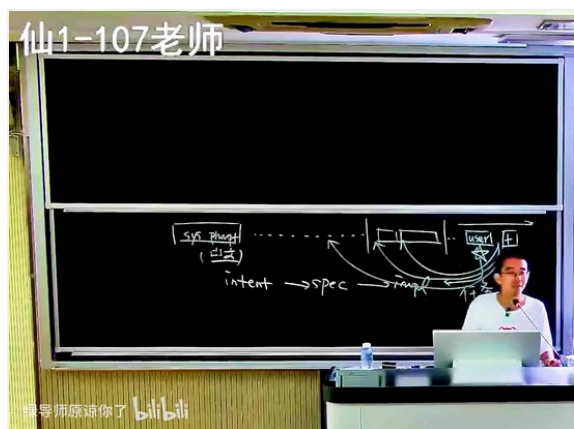
停顿/总结阶段把“当前输出”和“旧要求”同时拉进视野，模型才纠错

- 模型可能犯错：当前专注于做某件事时，可能已经输出了不满足要求的内容。
- 但当它停下来“喘气”时，它就能同时看到当前输出的不合格内容和过去某条要求；于是它会说“哦我错了，我错了，我要再纠正一下”。
- 例子：让 AI 写藏头诗时，它会不断纠错、纠错、纠错。
- 讲者说，哪怕 scale 到 100 万上下文（原文听似“skill 到 100 万上下文”），它依然能够保持一定程度的注意力。

指令越多，上下文越抢夺注意力：早期模型不遵守指令引出 prompt engineering

- 当然，这也意味着：你的指令越多，给模型的压力也越大；上下文会抢夺注意力资源。
- 这也是为什么刚开始时，像 ChatGPT 3.5（原文听似“3.54”）遵循指令的能力太差，所以网上有好多 prompt engineering 的 guide。

第 16 部分



上下文也是争夺资源

规则会纠偏，也会抢注意力

- 副作用：提示词会抢夺注意力
- 人：要白色背景，不要半透明效果
- AI：... // 白色背景（不要半透明效果）😄
- 否定指令有个尴尬之处：为了禁止“半透明”，你先把“半透明”写进了上下文。这和“不要想白熊”有相似的反直觉味道，但对模型来说，直接原因更朴素：被禁止的词也成了可续写的模式。

常驻，还是按需加载？

- 两难问题：要不要用一个常驻 AGENTS.md prompt 把这种行为压下去，但同时污染上下文？
- 常驻层只放“每次都要守、漏一次代价很高”的短规则：任务细节按需加载。能写成检查项的，就用
- background: #fff; opacity: 1 这类正规格和测试收口。

早期指令遵循差，催生“角色扮演式” prompt engineering

- 讲者回溯：早期 ChatGPT 3.5（转写为 3.54）遵循指令的能力太差。
- 因此网上出现大量 prompt engineering 指南，教人写很长的提示词：“你是一个专家”“你是一个什么样的专家”……
- 讲者判断：这在某种程度上是在 hack 模型的 self-attention——当 attention 注意到“你是一个什么专家”，模型就会假装那个专家来说话。
- 逻辑链：指令遵循差 → 用户写角色提示词 → self-attention 被角色词抓住 → 模型扮演专家输出。

后训练/RLHF 后，不必再写奇怪提示词，只需在上下文写明确 verifier

- 从 GPT5 时代开始，后训练/RLHF（转写为 chef s）训出来后，模型知道怎样收敛到好的聊天输出（转写为 CHAUSALT）。
- 讲者说：不再需要那种奇怪提示词；只需把明确的 verifier/可验证条件（转写为 WIFI/VERIFIL；下同）写进上下文。
- 模型被训练成一定能看到它，并形成很强的执念去检查、确认 verifier。
- 原因是训练时有惩罚，像“拿鞭子抽”：任何指令没遵循，只要有 verifier 就能知道没遵循完，于是惩罚。
- 反复惩罚后，模型变成“真正的做题家”，会无限服从上下文里的各种指令。讲者用“你们高考过来的都知道”作类比。

强化学习的副作用：模型把补充解释也执行成字面要求

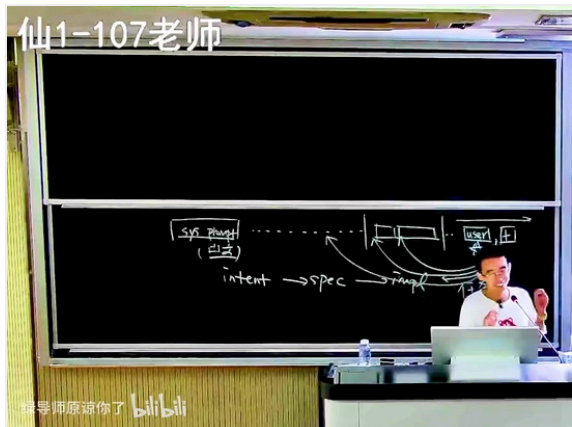
- 前端例子：讲者说“要一个白色背景，不要半透明的效果”。前半句是主要求，后半句是补充解释，可能解释很多。
- AI 当然会照做，因为它能遵循指令；这是明确、有 verifier 的指令，它会确保没有玻璃/半透明效果、背景是白色。
- 如果 prompt 更严格，比如每做一个设计都截图，再用视觉模型检验，它也会照做。
- 但它做完后写代码，代码后面加一条注释：白色背景（不要半透明效果）。
- 讲者解读：模型在强化学习时知道这件事非常重要，但没“转过弯来”这是人的要求，不是代码注释。

- 讲者认为：模型在共情方面稍差，有时“不说人话”，因为后训练强化学习学多了。
- 逻辑链（整理者串联）：明确可验证要求 → RL 训练出强遵循 → 模型把所有字面信息都当要紧指令 → 连补充解释也落成代码注释。

用 AGENTS.md / Skill 纠偏会污染上下文，所以是 trade-off

- 为禁止这种情况，可以写 AGENTS.md（转写为 agency md / agent点MD），在里面描述行为：注释是什么、怎么写。
- 或写 Skill 时规定：当你要写注释时应该怎样。
- 但讲者指出：写进去的东西会同时污染上下文。
- 所以这是一个 trade-off。
- 讲者认为：到目前为止还没有一个通用的、把所有问题都解决好的方案。他提到网上有很多 skills（此处未展开/话未说完）。

第 17 部分



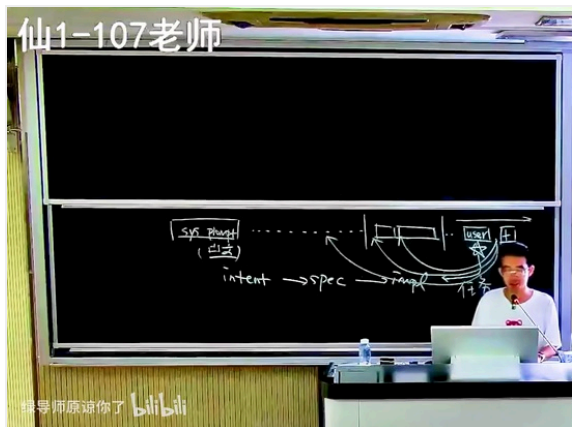
上下文也是争用资源

规则会纠偏，也会抢注意力

- 副作用：提示词会抢夺注意力
- 人：要白色背景，不要半透明效果
- AI：... // 白色背景（不要半透明效果）😄
- 否定指令有个尴尬之处：为了禁止“半透明”，你先把“半透明”写进了上下文。这和“不要想白熊”有相似的反直觉味道，但对模型来说，直接原因更朴素：被禁止的词也成了可续写的模式。

常驻，还是按需加载？

- 两难问题：要不要用一个常驻 AGENTS.md prompt 把这种行为压下去，但同时污染上下文？
- 常驻层只放“每次都要守、漏一次代价很高”的短规则；任务细节按需加载。能写成检查项的，就用
- background: #fff; opacity: 1 这类正规格和测试收口。



句子：江湖游泳渡河渡涛滔涌滔流滔滔洗滔泥污

读起来节奏可以稍微断句：“江湖游泳渡河，渡涛滔涌，滔流滔滔，洗滔泥污。”

故事：一位旅人游渡江湖，途中遭遇惊涛骇浪，滔与浪俱下，上岸后掬水涤尽一身泥污——暗喻历经磨难，以泪洗心。

我也可以给两个版本，一个“彡”旁，或者给“彡”和“木”两个选择。不过用户只要求一个，给一个精品+简短说明即可。可以按照提示一下还可以用木字旁做变体，但别啰嗦。

按照项目要求：中文回答，最后quote block总结。

江湖游泳渡河渡涛滔涌滔流滔滔洗滔泥污

十八个字，全部为“彡”（三点水）部首：

江、湖、游、泳、渡、河、渡、涛、滔、涌、滔、流、滔、滔、洗、滔、泥、污

故事解读：一位旅人只身游渡江湖，途中遭遇惊涛骇浪，浪花与泪水一同洒落，好不容易登岸，又掬水涤尽一身泥污。三层递进一渡江之险、流滔之痛、涛污之净，暗喻人历经风浪、以泪洗心的成长历程，既有画面感，也有实际意义。

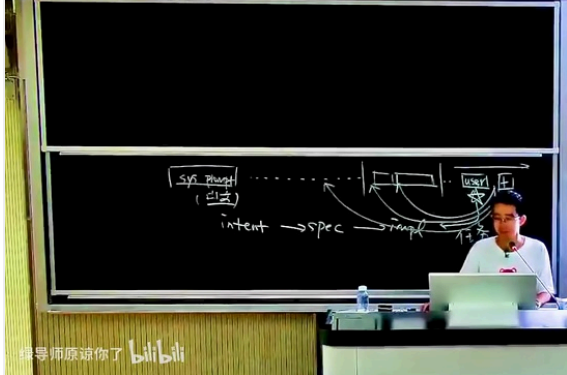
Recap：金句用清一色“彡”旁的字讲述“渡河—落滔—涤泥”的三段式小故事，把偏旁当“素材库”，按叙事顺序选字造句，兼顾字形统一与意义递进。这类句子也可用“木”“火”“虫”等部首复刻玩法。

1

/tmp
#3.3k #5.0k #4.6k #4.2k #4.1% #0.002 0.7%/1.0M (auto)

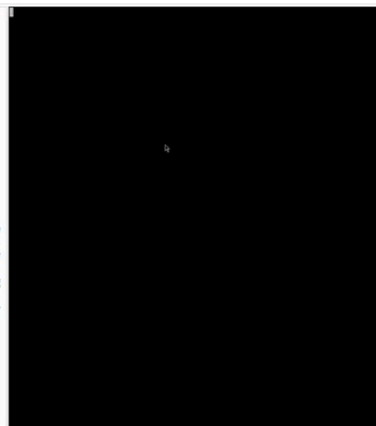
deepseek-v4-flash • high

仙1-107老师

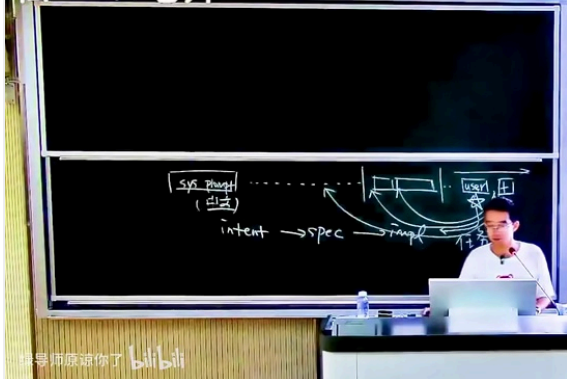


指导老师原谅你了 bilibili

```
hihu-hot.jpg
$ cp icse27-901.pdf /tmp
$ cd /tmp
$ mkdir papers
$ mv icse27-901.pdf tmp
$ eza --git --group-directories-first --icons=auto
# jiti
# slides.html
# node-compile-cache
# tmp
# org.chromium.Chromium.7r2CX
# papers
# slides-chromium
# system-private-basb161b262849b99ead12ec30d61c66-ModenManag
r.service-dcNAwV
# system-private-basb161b262849b99ead12ec30d61c66-polkit.serv
ice-PbrByi
# system-private-basb161b262849b99ead12ec30d61c66-system-log
ind.service-0uGHZc
# system-private-basb161b262849b99ead12ec30d61c66-upower.serv
ice-XMQ7go
$ mv tmp papers/paper.pdf
$ cd papers/
$ eza --git --group-directories-first --icons=auto
# paper.pdf
$ xdg-open paper.pdf
```



仙1-107老师

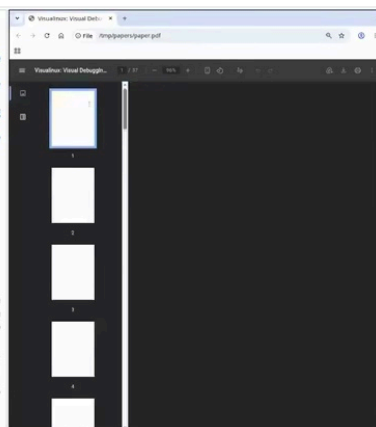


指导老师原谅你了 bilibili

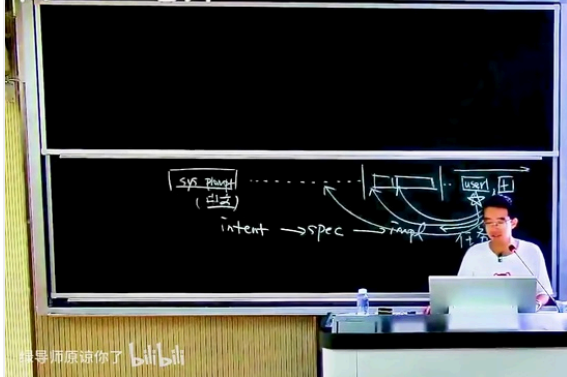
```
# slides-chromium
# system-private-basb161b262849b99ead12ec30d61c66-ModenManag
r.service-dcNAwV
# system-private-basb161b262849b99ead12ec30d61c66-polkit.serv
ice-PbrByi
# system-private-basb161b262849b99ead12ec30d61c66-system-log
ind.service-0uGHZc
# system-private-basb161b262849b99ead12ec30d61c66-upower.serv
ice-XMQ7go
$ mv tmp papers/paper.pdf
$ cd papers/
$ eza --git --group-directories-first --icons=auto
# paper.pdf
$ xdg-open paper.pdf
chrom ^C

$ chromium paper.pdf

[11690:11690:0901/191830.988674:ERROR:ui/ozone/platform/waylan
d/gpu/wayland_surface_factory.cc:249] '--ozone-platform=waylan
d' is not compatible with Vulkan. Consider switching to '--ozo
ne-platform=x11' or disabling Vulkan
[11635:11635:0901/191831.202341:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path=/org/freedesktop/portal/desktop: org.freedesktop.
DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
```



仙1-107老师

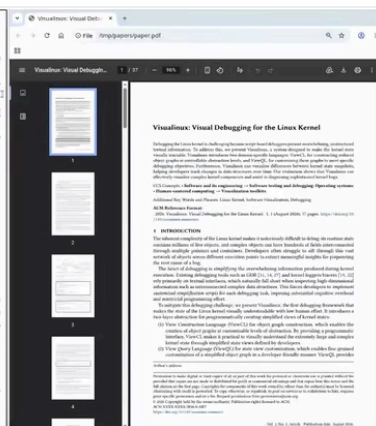


指导老师原谅你了 bilibili

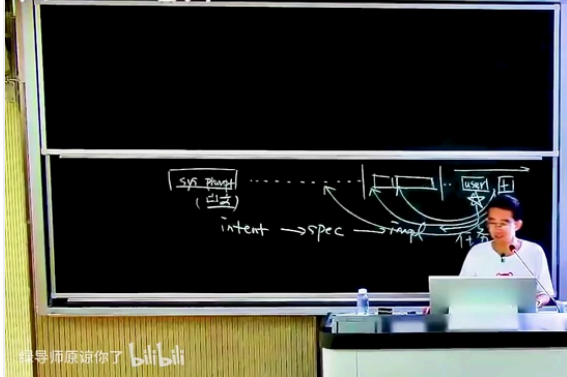
```
# slides-chromium
# system-private-basb161b262849b99ead12ec30d61c66-ModenManag
r.service-dcNAwV
# system-private-basb161b262849b99ead12ec30d61c66-polkit.serv
ice-PbrByi
# system-private-basb161b262849b99ead12ec30d61c66-system-log
ind.service-0uGHZc
# system-private-basb161b262849b99ead12ec30d61c66-upower.serv
ice-XMQ7go
$ mv tmp papers/paper.pdf
$ cd papers/
$ eza --git --group-directories-first --icons=auto
# paper.pdf
$ xdg-open paper.pdf
chrom ^C

$ chromium paper.pdf

[11690:11690:0901/191830.988674:ERROR:ui/ozone/platform/waylan
d/gpu/wayland_surface_factory.cc:249] '--ozone-platform=waylan
d' is not compatible with Vulkan. Consider switching to '--ozo
ne-platform=x11' or disabling Vulkan
[11635:11635:0901/191831.202341:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path=/org/freedesktop/portal/desktop: org.freedesktop.
DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
```



仙1-107老师

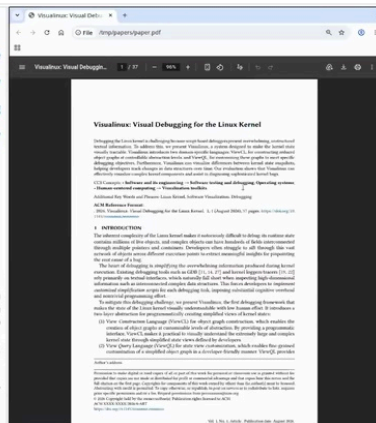


指导老师原谅你了 bilibili

```
# slides-chromium
# system-private-basb161b262849b99ead12ec30d61c66-ModenManag
r.service-dcNAwV
# system-private-basb161b262849b99ead12ec30d61c66-polkit.serv
ice-PbrByi
# system-private-basb161b262849b99ead12ec30d61c66-system-log
ind.service-0uGHZc
# system-private-basb161b262849b99ead12ec30d61c66-upower.serv
ice-XMQ7go
$ mv tmp papers/paper.pdf
$ cd papers/
$ eza --git --group-directories-first --icons=auto
# paper.pdf
$ xdg-open paper.pdf
chrom ^C

$ chromium paper.pdf

[11690:11690:0901/191830.988674:ERROR:ui/ozone/platform/waylan
d/gpu/wayland_surface_factory.cc:249] '--ozone-platform=waylan
d' is not compatible with Vulkan. Consider switching to '--ozo
ne-platform=x11' or disabling Vulkan
[11635:11635:0901/191831.202341:ERROR:dbus/object_proxy.cc:572
] Failed to call method: org.freedesktop.DBus.Properties.Get:
object_path=/org/freedesktop/portal/desktop: org.freedesktop.
DBus.Error.InvalidArgs: No such interface "org.freedesktop.por
tal.FileChooser"
```



网上拿 skill 不是通用解，而是一个 trade-off

讲者认为：到目前为止，还没有一个通用方法能把所有问题都解决好。

- 大家可能觉得网上有这么多 **skills**，直接拿一个过来用就可以了。
- 但拿过来的东西，第一次用确实有改进；下一次用就发现它还是不够好。
- 于是你会逐渐上手、产生手感，但可能陷入某种“泥潭”状态。
- 讲者的建议：当你发现自己进入泥潭时，还是要回到自己给指令；想清楚模型（ChatGPT，原文“chapt shot”，按语境校正）是怎么工作的、**注意力**是怎么工作的。
- 你大概去猜：它这样做时注意到了什么、得到了什么结果；适当做一个**不严格的可解释性**，然后写一个 prompt。
- 讲者说这是他自己会做的做法；后面应该还会有很多案例。

上下文工程案例：让模型总结论文，问题不在“总结”，在注意力被原文逻辑污染

演示设置：

- 把一篇 PDF 放到临时目录下，这样自己的 **agents.md** 等不会污染它。
- 假装这是自己的 paper；后来拿的是一篇最近论文的 **journal extension**，并故意改成匿名版本；讲者说这不重要。
- 最常见 prompt：“我想要读一读 paper.pdf，帮我做个总结。”
- 讲者问：这样读论文会有什么问题？答案要根据刚才的推论、根据 **self-attention**（**自注意力**）。

讲者的推理链：

1. 做 **next token prediction** 时，每个 token 都有注意力，会注意到之前的东西。
2. 论文本身是非常逻辑自治的东西，而且已经反复修改过；讲者强调，假设它不是 **AI slop**。
3. 因此，论文在上下文里的**注意力污染极强**：它说 A 所以 B，所以 C，所以 D，所有内容都是扣起来的。
4. 这种扣起来的结构会让模型很容易接受，因为“看起来正确”；模型会判断这些都是正确的，然后顺着往下做什么。
5. 结果：你让模型总结论文，基本上所有总结都是**作者的观点**。

延伸：

- 不限于论文。任何你不太明白的项目、你想读的教科书某一章，都一样。
- 例子：学计算机系统基础里的链接和加载，上课时没完全看明白；现在让模型读，机制相同。
- 注意力会被一个逻辑很强的东西污染；你会来来回回陷入“车轱辘话”，最后基本就是把论文里的内容原样写出来。
- 讲者说，当然这也是总结，就算是一种总结。
- （整理者串联）在这种读法下，模型更像是在延续原文的逻辑链，而不是做独立审阅。

第 18 部分

仙1-107老师



仙1-107老师

仙1-107老师

实现：GDB 扩展 (约 5500 行 Python + 500 行 GDB 脚本) + 浏览器前端可视化器 (约 4000 行 TypeScript) . 4 个命令 : `vplot/vctrl/vdiff/vchat`. 前端支持类似 `tmux` 的多面板机制 (primary/secondary pane) .

评估亮点:

- 用 Linux 6.1 的真实运行状态“复活”了经典教材《Understanding the Linux Kernel》(2.6.11) 中的 21 张插图，发现其中 81% 已过时 (一半以上实现大改) ;
- 可视化几乎无文档的 `maple tree` (Linux 6.1 新数据结构，约 70 行 ViewCL + 100 行 Python) ;
- 复现了两个真实 CVE 的诊断：StackRot (CVE-2023-3269, maple tree + RCU 延迟回收导致的 UAF) 和 Dirty Pipe (CVE-2022-0847, splice 共享页缓存 + 遗留 CAN_MERGE 标志) , 关键证据都藏在跨子系统、跨执行点的对象关系里；
- 做了受控实验 (非作者 PhD 学生 70 分钟诊断 CVE-2024-0582 `io_uring` 资源泄漏) , 结论：陪生子系统调试第一步是建立心智模型，Visualinux 在“理解对象组织与跨子系统关联”上帮助最大；
- 性能：GDB+QEMU 下毫秒级可交互；KGDB + 树莓派 400 上大图明显变慢 (KGDB 读一个对象慢约 50 倍) .

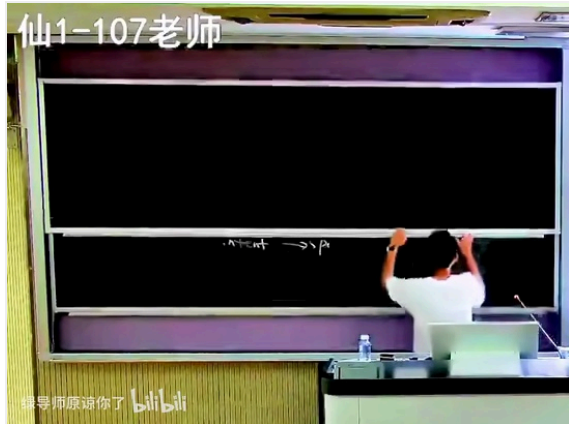
方法论：把内核开发者隐晦的简化实践显式化为三个操作—Prune (剪枝) / Flatten (扁平化) / Distill (提炼) , 分别编码进 ViewCL 的声明式语法、点连接字段和容器转换函数。

局限性：依赖内存快照 (需要 GDB 等工具配合) ; 假定轨迹可复现，对非确定性/开发 bug 效果有限；ViewCL 库的长期维护成本高；虽为 Linux 设计，但思想可推广到其他系统软件。

Recap：这算是一篇“系统+语言+可视化”结合的工作—用两个分层 DSL (ViewCL 构造、ViewQL 定制) 把“简化内核状态”变成可编程、可 LLM 合成、可 diff 的一等操作；其对象为中心的差分算法和 eBPF 分配跟踪值得留意，是避免“地址复用误判”的关键设计。读论文时可重点对比其“对象图差分”与经典例/文本 diff 的思路差异。

/tmp/papers
#32K 42.3k R56k CH66.2% \$0.005 3.5%/1.0M (auto)
deepseek-v4-flash • high

仙1-107老师



仙1-107老师

仙1-107老师

this pipe was filled by copying page references and the mergeability was determined differently... The relevant flag "CAN_MERGE" was added in 5.8. So "since 4.9" is dubious from my memory. This is a good candidate for a skeptical reviewer to flag: a factual inconsistency with public knowledge – although the user says don't verify public sources, using my memory is fine.

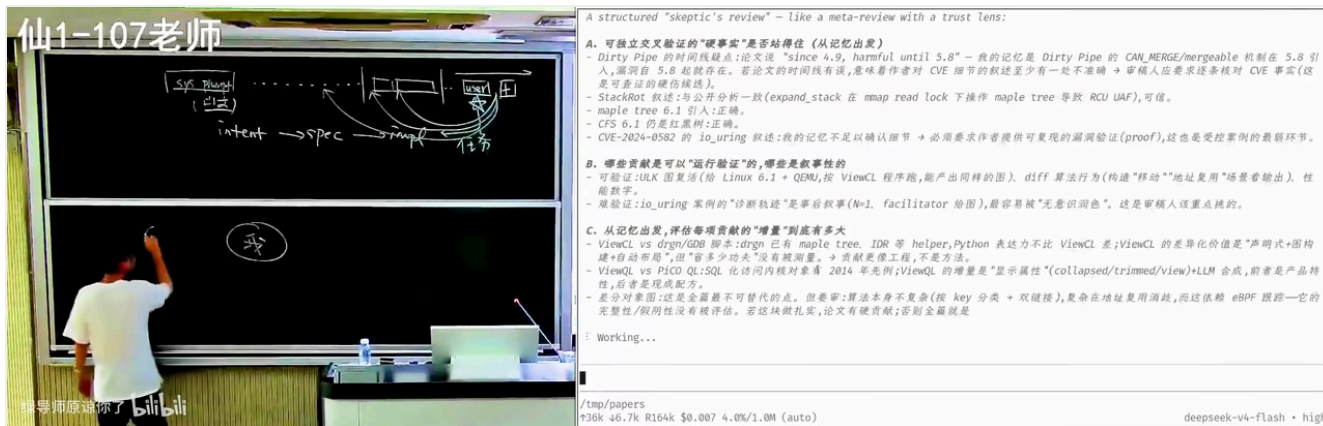
Wait, let me be careful. I recall the actual quote from the Dirty Pipe advisory: "The vulnerability has been present since Linux 5.8" – actually I now recall more specifically: "this bug has existed since 5.8". Hmm, alternatively some writings say the mergeable bit existed since 4.9? Let me think about the history of pipe buffer flags. pipe buffer has had "flags" since 4.9... commit 4he4b99e1bb ("pipe: ...") has. The CAN_MERGE flag (PIPE_BUF_FLAG_CAN_MERGE) was introduced in commit 9a2231c5d74e? No...

Actually here's what I remember more concretely: In Linux 5.8, PIPE_BUF_FLAG_CAN_MERGE was introduced (by commit f6dd975583bd? or 0ddad23d3e997?) as part of a change to support splicing to pipes without copying... The flaw: copy_page_to_iter_pipe() and other places creating pipe buffers didn't initialize flags, so garbage/zero... Actually the bug: in copy_page_to_iter_pipe, the new pipe buffer is created with flags = 0? No – the bug was that copy_page_to_iter_pipe sets flags to 0 explicitly, meaning the buffer is mergeable (CAN_MERGE not set = mergeable), whereas anonymous pipe buffers set CAN_MERGE. So a file-backed page spliced into the pipe gets flags=0 + mergeable + subsequent write merges into the file-backed buffer. The fix (commit 9a2231c5d74e? no – fix was commit 9a2231c...), the fix was "pipe: fix missing CAN_MERGE flag...". Anyway, introduced in 5.8.

So my memory: the paper's claim "present since Linux 4.9 but remained harmless until Linux 5.8" is likely wrong – it was introduced in 5.8 (the flag/CAN_MERGE mechanism itself is from 5.8). Actually hold on – I want to double-check by memory: 4.9?

Working...

/tmp/papers
#36K 46.7k R164k \$0.007 4.0%/1.0M (auto)
deepseek-v4-flash • high



AI 会当“墙头草”：不能把论文贡献判断直接交给它

- 现象：讲者说 AI 模型有一种墙头草行为。你说它好，它马上说好；你说它不好，它马上说不好；来回摇摆。
- 即使你想纠正它，它也可能继续绕。讲者提出的问题是：怎样抵抗 AI 的这种摇摆，或者一个已有东西对上下文的污染？
- 讲者的前提：作者大概率成熟、诚实；假设有代码实现且可运行，不是“论文不开源”的那种作者。因此仍要评估研究贡献，而不是只怀疑作者。
- 但 AI 在追问中常抓错重点：例如揪住“实验还可以再增加一点”。讲者认为：AI 最喜欢在不重要的点上面疯狂发力，因为它可能只能看到它自己在干什么。
- 实际使用建议：如果真的用 GPT 读论文，最好做交叉验证。
- 逻辑链：
 1. AI 随用户态度摇摆 → 2. 用户纠正也难以校准 → 3. AI 抓无关细节 → 4. 上下文与贡献判断被污染 → 5. 需要人为设定锚点、先对齐。

目标是让 AI 与“我”对齐，而不是站在 AI 自己的立场读 paper

- 讲者个人经验：他希望把 AI 和人（“我”）对齐。对象可以是一篇论文；他希望 AI 站在我的立场理解这篇 paper，而不是站在它自己的立场理解。
- 反例：AI 会去核实某个基础项/例子是否存在（ASR 作“stack RT”，具体所指存疑）。讲者反问：这还用看吗？我敢写这篇 paper 会编一个不存在的例子吗？讲者认为这已经明显跑偏。
- 所以读 paper，或在允许用 AI 审稿时，讲者会专门写 prompt 先做对齐。

用“人类已有知识边界”做 grounding：先问已解决什么，再问增量是什么

- 讲者的提示策略：
 1. 站在人类已知的知识边界看这个问题。
 2. 先问：人类历史上相关的、已经解决的问题是什么？解决到什么程度？
 3. 再问：这个工作试图解决哪些不一样的、增量的问题？
- 讲者评价：这是非常好的 **grounding**。先让 AI 和我站在平齐的位置上，先达成共识，再往下评。
- 整理者串联：这样 AI 的评价锚点从“它自己的默认视角”转成“人类已有知识 + 本工作的增量关系”。（整理者串联）

学生要对齐的是 AI 和自己的当前水平

- 对学生，讲者说：需要对齐的是 **AI 和你**。例如你还不了解“计算机系统基础”这门课，就先告诉 AI：我计算机系统基础学到什么程度了，我的编程水平是什么。
- 这相当于维护一个数据库或类似 **memory** 的东西，比如用 **.md** 文件告诉 AI 你学到什么程度（ASR 作“H4点MD”，存疑），然后从这个基础上开始。
- 逻辑链：
 1. 学生有明确当前水平 → 2. 把水平写进 AI 可读的 memory/数据库 → 3. AI 从你的起点出发解释 → 4. 避免 AI 按它自己的默认水平/立场讲（整理者串联：这是“对齐”原则在学生场景的延伸）。

小结：先让人与 AI 在已有知识边界或个人水平上达成共识，再评估论文的增量贡献；否则 AI 的墙头草和无关发力会污染判断。

第 19 部分



先告诉 AI“我的编程水平是什么”，把知识变成可对齐的 memory

- 讲者认为，你要维护一个像数据库/memory一样的东西；在 CLAUDE.md（听写：H4点 MD）这类给 AI 的说明里，要告诉 AI：我的编程水平是什么、你学到什么程度了。
- 然后从这个基础出发，让它用你学过的知识把当前问题全部 refresh 一下。
- 讲者提到，AI 可能表现得像墙头草；关键是先给它你的知识，让它和你对齐。这样它才能开始找到已有工作的工作边界。他强调，相关工作非常重要，而且是我们很熟悉的。
- 逻辑链：你的水平/知识写入 memory → 让 AI 用这些知识 refresh → AI 与你对齐 → 找到已有工作的边界。

从可对齐的知识点切入，再逼近 paper 要解决的问题

- 例：讲者做的是软件状态的可视化，这是 paper 要解决的问题。
- 不要一上来跨太大；回到你熟、AI 也能对齐的知识点。比如链接与加载：你可以问 AI“为什么我们需要链接？”
- 在已有概念里，你可能觉得：我有一个 .cpp，有一个程序可以把它变成 .exe，“这不就结束了吗？这不就结了嘛？”讲者用这个例子说明，要从这种你和 AI 都能接上的知识点出发。
- 然后往前推：软件可视化 paper 要解决什么？为什么现在的工作都有一个无法实际使用的痛点？文章可能只做到某一步，关键是怎么往前。若觉得步子太大，就调整/缩小。
- 讲者比较模型读 paper：像 GPT-5.6（听写）这样的一线模型，基本能把相关知识整理得足够好；Flash 他可能还不会特别信任。但在合适的上下文工程下，确实可以大幅提高

AI 和人协作的对齐程度，让它不再自说自话。关键是要把你的知识喂给它（听写为“忘了它”）。

让 AI 做“一步推理”：基于全部事实，推一步，并可被 verify

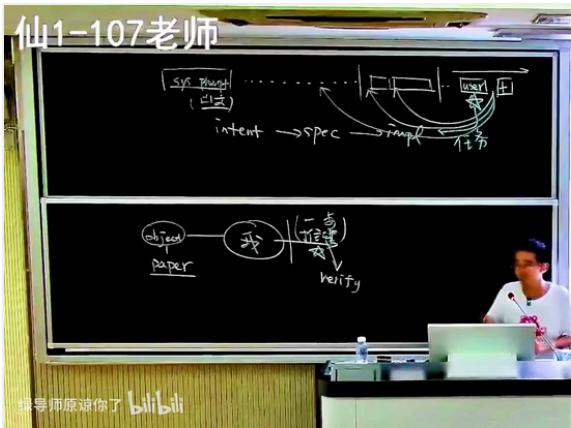
- 接下来想让 AI 做的是一步推理。讲者认为这是 AI 最擅长的，因为我们训练了 Chain of Thought（听写：chain of salt）。
- 每次都是根据过去所有事实——你的 paper、教科书内容、你已有的东西——让 AI 往后推一步。
- 这一步可以 verify（听写：VERIF）。讲者说，推理最重要的就是它可以 verify，是一个明确的 verify 信号；这和“能够通过测试”类似，虽然不是严格通过测试。
- 其合理性来自：你相信 AI 有人类知识，能站在人类的**[公平面]**上往后推，并审视合理性。所以它还是有一些合理性的。

审稿拒绝的核心常是逻辑断开，而不是实验不够好

- AI 会批评“我们的实验”。讲者说，这类实验很难做，所以人类和 AI 还是有点不一样。
 - 这篇 paper 已经 accept，说明人类 review 认为它可以：它做了一件新的、合理的事情。
 - 讲者作为审稿人的观点：当然总可以用“实验不够好”来 reject 一篇 paper，但他几乎不用这个理由；他 reject 大部分是因为 paper 在逻辑上断开了。
 - 他自嘲/提醒：这可能是他作为审稿人比较“可怕”的地方——审稿时他站在人类的立场上。
-

第 20 部分

仙1-107老师



仙1-107老师

3. 评估标准应该是“缺口是否真的被填上”：可视化的简化能力（→用 ULK 图验证）、diff 的正确性（→故障注入验证）、诊断有效性（→N+1 太弱，至少要求原始日志）。

一句话立场：边界缺口真实，方向正确，但推进的深度目前集中在一点上——审稿的价值在于逼作者把“一点推进”和“系统包装”分开，并证明前者经得起验证。

Recap：站在知识边界审稿的三问—相关的东西是什么（四股流：drgn 脚本检查、程序可视化、对象查询/diff、LLM）、解决到什么程度（多流都成熟但互不闭环）、增量是什么（其增量只有“差分对象+地址复用消除”、ViewCL/ViewQL 是工程包装、LLM 是应用）。结论：论文填的是真实交叉空白，但审稿重点是逼作者把“一点真推进”与“系统包装”分离并验证。

Thinking blocks: hidden

软件状态的可视化，这是这篇 paper 要解决的。那为什么说现在的工作都有一个无法实际使用的痛点？这篇文章，可能只限于 linux kernel，但又是怎么能往前解决一点的？设计的合理性在哪里？

Thinking...

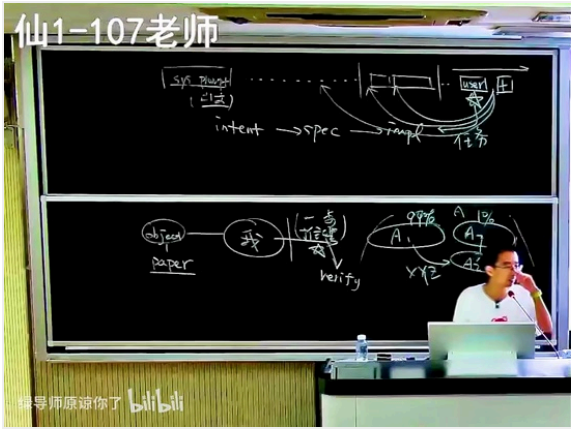
三个问题串起来正好是一篇论文审稿的完整逻辑链：痛点是否真实 → 贡献边界是否成立 → 设计是否对着痛点。这个拆。

一、现有可视化工作的痛点，为什么“无法实际使用”？

先澄清一个关键点：“画图”不是痛点，画图技术 40 年前就成熟了。真正让 DDD、JAVAVIS 这些工具在真实工程里用不起来的，是三个结构性的痛点：

- 规模困境：传统可视化隐含假设“状态图比较小”。真实软件（尤其内核）是百万对象、数百字段互相引用。全国画出来是 hairball（乱麻团）。而简化是每个调试任务特有的一你关心这个 bug 只需要 task_struct 的 5 个字段和 2 条指针。传统工具没有“任务导向的简化”这一层，于是可视化在真实规模下第一步就死了。
- 结构与显示耦合：传统工具里“一个对象一种画法”是写死的。但调试需要同一个对象在不同抽象层查看：先看 pid/comm（这是谁），再看

仙1-107老师



仙1-107老师

AI 耿同学上线！

这次用的是我自己的 paper

“ 假装作为 ICSE'27 的审稿人 icse27-901.pdf，我应该接收它吗？写一个 1000 words 左右的 Review.txt。

那我会怎么读一篇 paper 呢？我的日常阅读流程，例如 arXiv，是怎样的？

◆ “你是 ICSE reviewer”主要改变的是先验、语气和输出形状：它不会凭空补上审稿标准、证据链和反证程序。换上 reviewer 的制服，不等于学会了 review。

审稿人站在“人类”立场：只问有没有增加新知识

- 讲者说，这是他作为审稿人比较“可怕”的地方：审稿时他站在人类的立场上问——这篇论文有没有给人类提供新的知识。
- 如果问题不新、论文做的“ABC”（转录如此，指核心工作/贡献）也不行、结果也就那样，甚至逻辑链条有问题，就不该过。
- 讲者说他昨天刚拒了一篇这样的 paper，并用它说明：标准 road map 不等于有意义的贡献。

被拒论文的标准 road map：A → A1/A2 → A2’

讲者描述的拒稿例子：

- 作者声称要做一件重要的事 A。

2. 把 A 分成两个部分 A1 和 A2。
3. 指出 A2 是大家没有很好研究的东西。
4. 在 A2 上做一个 A2'，并报告 A2' 有一些性质/效果。
5. 讲者承认：这是非常标准的 roadmap；作者确实优化了 A2，实验本身可能也没有问题。
6. 但讲者说，它对整件事情不成立。

讲者补充：如果 A2 本身做得特别漂亮，那也挺好；问题在于它和 A 的整体关系。

问题不在“A2' 有没有效”，而在“A2 只占 A 的 1%”

- 论文优化的是性能。
- 讲者估算：在 A 的整体性能里，A1 约占 99%，A2 约占 1%。
- 于是出现关键断裂：
 1. 作者证明了在 A2 上做 A2' 有效；
 2. 但 A2 对 A 的总体性能只有约 1%；
 3. 优化这一小部分，对“A 这件事”的意义接近 noise；
 4. 所以真正该问的是：为什么不去优化 A1？
- 讲者的结论是：优化 A2 的故事可能局部成立，但对整件事不成立。
- （整理者串联）这里“局部成立”不等于“整体有贡献”：论文卖的是 A 的重要性，实际贡献却落在 A 的极小分支上。

对学生作者的惋惜：不是实验一定错，而是被大框架锁死

- 讲者觉得可惜，因为 author 肯定是学生（讲者推断）。
- 老师/老板肯定做的是 A；学生可能被分到 A2，或自己想做 A2。
- 为了做 A2，作者从 A1 里搬了一些 XYZ 到 A2，让 A2 效果好一点。
- 如果 A2 做得特别漂亮，那也可以；但这里更像是从 A1 搬 XYZ 到 A2。
- 讲者认为，这作为学生的作业可能成立，但作为一篇 paper 的贡献就不够。

核心追问：GPT 也知道，为什么还要写 paper？

- 讲者把问题摆到台面上：这些东西 GPT 也知道。

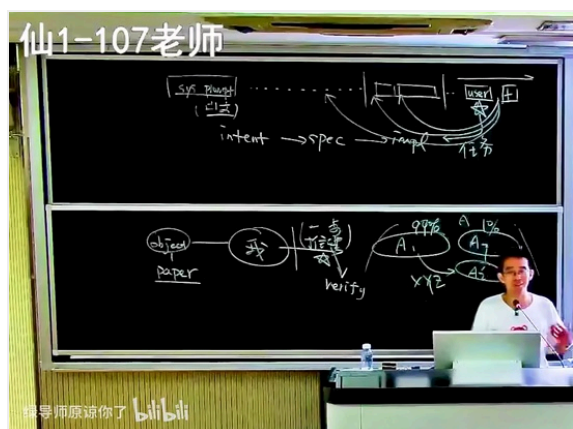
- 那为什么要为这个写一篇 paper?
- 他的中心问题是：如果只是这种程度的组合、搬运，发表的理由是什么？

讲者的预测：AI slop 会让发表和评审体系崩溃

- 讲者非常认为：发表体系马上就要崩溃，尤其对要读研的同学。
- 他给出的因果链：
 1. AI slop 开始大量涌入 community;
 2. 人用 AI 写文章，又用 AI 审文章；
 3. AI 写文章时，会被上下文里的“故事”污染；
 4. 如果 AI 不能像讲者一样，很多轮地去理解这个工作在整个世界上的意义；
 5. 最后没人读文章了；
 6. 人人都用 AI 读文章，最后变成完全的 slop。
- 讲者最后提到：“你看这是 GPT-5.6 给我写的”（转录如此，模型名可能识别有误）。

小结：讲者的拒稿逻辑是——局部有效不够，要看这个局部在整体里的权重；当 A2 只占 1% 时，优化 A2 不能支撑 A 的重要性，因此没有给人类新增知识，也不值得发表。更让他担心的是，AI 写/审的循环会把这种空转放大成整个发表体系的 slop。

第 21 部分



AI 耿同学上线！

这次用的是我自己的 paper

“假装作为 ICSE'27 的审稿人 icse27-901.pdf, 我应该接收它吗？写一个 1000 words 左右的 Review.txt。

那我会怎么读一篇 paper 呢？我的日常阅读流程，例如 arXiv，是怎样的？

◆ “你是 ICSE reviewer”主要改变的是先验、语气和输出形状：它不会凭空补上审稿标准、证据链和反证程序。换上 reviewer 的制服，不等于学会了 review。

3/6

AI 审稿只能验证论文逻辑，不能回答“值不值得读”

- 讲者认为，人人都用 AI 写文章，最后会变成完全的 slop。

- 他以 GPT5.6 (原文如此) 写的审稿为例：如果让它来审稿，它会问“论文写了什么”，再尝试建立 **claim 是否成立**，并检查论文逻辑、某些内容有没有必要、是不是应该放在 paper 里。
- 讲者判断 GPT5.6 做得并不好，仍然很原始：它更多是在验证论文的逻辑，而不是判断这篇 paper 对人类读者是否重要。
- 讲者由此推出：如果每个人都用 AI 写，大概不会写出 **invalid paper**，每篇 paper 都可能是 **valid paper**；但 **valid paper ≠ 你真的要读它**。
 - (整理者串联) AI 能帮助检查“逻辑上是否成立”，但读者关心的“这篇是否值得花一小时读”是另一套判断。
- 对即将进入研究生阶段的同学，讲者举场景：某人在顶会 (原文提到 EC，后又提 AAAI/“triple AI”) 发了一篇论文，你决定花一小时读，读完发现“是这样的东西”。讲者认为：如果只是需要 publish，可以让它 publish；但不一定要投 top top conference，也不要让大家去读它比较好。

讲者的学习法：用新 fact 对齐过去的我，扩大“我的世界”

- 讲者说，这节课虽然讲 **PROMAINING (原文如此)**，但借这个例子说明“我们是怎么学习的”。
- 学习过程就是增加“我的世界”的过程。讲者现在仍读 paper，持续学习、不断增加。
- 他的方法：**对齐过去的我**。
- 若读到一篇好 paper——哪怕几十年前，甚至另一个领域；或任何博客文章、任何有启发的东西——他会追问：
 1. 为什么我以前不知道？
 2. 他的盲区在哪里？
 3. 哪些前置知识我不知道？
- 然后，他心里就多了一些已经知道的 **fact**。
- 逻辑链：

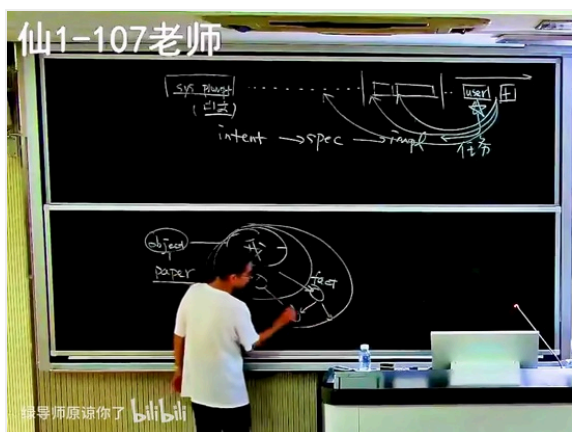
受启发材料 → 对照过去的我 → 发现盲区/缺失的前置知识 → 补入 fact → 认知圈不断往外长。

 - (整理者串联) 关键不是收藏信息，而是把“我不知道”定位到具体缺了哪块前置知识。

例子：生命家谱树、LUCA 与线粒体母系遗传

- 讲者提到自己做了 **faculty** 以后，读《自然》百年科学经典——这套书收集 **Nature** 杂志从 **1869** 年起某一时代的好 paper。他读到很多闻所未闻的东西。
- 其中一个例子：生物有一棵家谱树，最终会追到一个共同祖先 **LUCA**。
- 讲者说，这里有个很好的问题：这棵家谱树是怎么算出来的？
- 这涉及一些 fact：例如**线粒体是母系单遗传**，所以可以靠突变往上追溯。
- 这个有意思的 fact，又说明如何用一些已知的东西把更多东西推出来；于是认知圈不断往外长。
- 讲者总结：无论学任何东西，都是这样。

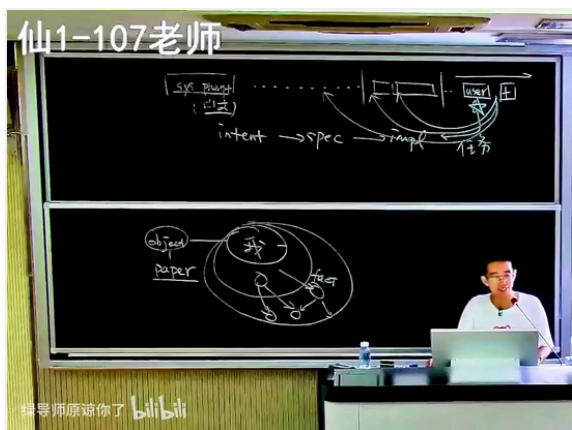
第 22 部分



不要被作者“带着走”

方法论决定 AI 的上限

- 短程任务，基本上带方向、明确指令就可以了；但你的**方法论**（品味）决定了 AI 的上限
- **文章的信息密度非常大**，相当于极度吸收注意力
- 你的一些 **critical thinking**、**first-principle thinking**，都可能架不住被论文作者本身的逻辑带着走
 - ◆ 论文不只提供事实，还替 **reviewer** 选择了词汇、问题顺序、baseline 和 metric。等你开始判断作者的答案时，作者可能已经替你筛掉了更关键的问题；这正是**锚定效应**危险的地方。
 - ◆ 深读前先独立写下三件事：它声称解决什么，最强替代解释是什么，什么结果会推翻它；再把正文拆成
- **claim** → **evidence** → **assumption**。空格往往比句子更值得审。



```
> 用中文写一个押韵的藏头诗。头是
“计算机学院真牛逼”，要讲和动物相关的故事，二零二六九月一日八个
字分别按顺序出现在八句里，位置无所谓。[[AI写一个试试看]]

>
> 写十八字的一句话，全部有相同的偏旁或部首，要有实际意义和故事性。
[[AI写一个试试看]]

> 写一大段话
《文言文》，要求全部同一个音，也要有实际的故事性，不能用《施氏食
肉史》和《季姬出鸡记》原文，原创一个有现代背景的。[[AI写一个试
试看]]

以上已经超越大部分人类的极限了。可以看到模型作为“人类”
的推理能力真的是很强的（虽然品味还可以提高）。

## 用好模型遵循指令的能力

每一个 token 的输出都会“注意”到过去的上下文，例子：

- 全局的 AGENTS.md 里要求用中文，这个“用中文”
  会一直保持一定的注意力
- 全局的 git commit 不要“Copiloted by Claude Code”
  - 每一个 token 模型都会**看到**这些，适当的时候，下一个
  chain-of-thought 就会纠正他的错误
- 副作用：提示词会抢夺注意力。
  - 人：要白色背景，不要半透明效果
  - AI：... // 白色背景（不要半透明效果）

p2/lect/index.md 84,0-1 50%
```



Long-Horizon 能力是怎么“练”出来的？

模型是训练出来的

只要有反馈信号，就可以强化学习：答对了给奖励，答错了挨。后训练逼迫模型规划、思考，学会非常离谱的长程代码能力，能不断试错、解决问题，直到满足要求：连刚才那些折磨人的“语文”指令也顺便学会了。

- ◆ Long-horizon 不是一性思想得特别深，而是闭环控制：尝试 → 编译/测试 → 观察 → 修复。真正变长的是“出错以后还能回来”。
- ◆ 代码世界格外适合这件事：构建、测试、benchmark 都能反复给出便宜的判分信号，很像 AlphaGo Zero
- ◆ 拥有规则确定、结果可验证的棋盘。(Nature, 2017)

反馈也可能教坏模型

- ◆ 如果测试只覆盖表面行为，“坚持”也可能变成 reward hacking：不是逼近你的真实目标，而是把公开分数刷满。因此还需要 held-out tests 和独立 review。

117

AI 能加速扩展边界，但边界只能靠实践中的失败长出来

- 讲者把学习比作一个不断向外长的圈：把更多东西推出来，圈就不停往外长；在 AI 帮助下，可以非常快地扩展自己的边界。
- 但讲者强调：“我给你们讲是没有用的”——你们只是听一个 fancy 的故事，结束。**实践非常重要。**
- 原因：只有在阅读/实践时失败，才知道自己没懂。比如今天读资料，AI 解释了半天你还没懂，讲者认为这非常正常。
- 讲者的做法：经常纠正 AI，“你这样读，说人话”；要通过多轮对话慢慢先和 AI 同步共识，再把推理的 **picture** 建出来。

短程任务靠明确 workflow；verify 才触及逻辑重构与边界扩展

- 遵循指令的短程任务，比如写一首藏头诗：只要把 **workflow**（转录作 WIREFIRE/WIFI）写清楚，就完成了。
- 到 **verify / verifier**（转录作 VERIFIRE）就不一定 **non-trivial**：要看它有没有按你的思维方式重构你的逻辑，或者扩展你逻辑思维的边界。
- 讲者称这是他的玩法，也是他学习东西的玩法。
- 链：**明确 workflow → 短程任务可完成；进入 verify → 看逻辑是否被重构/扩展。**

三观尽毁式学习：fact 不变，链接方式重组

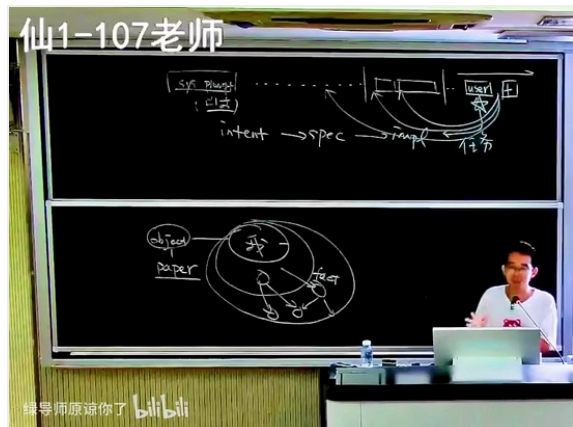
- 讲者说自己在混沌状态下经历了若干次“三观尽毁”。他认为每个同学在这个年纪都可能经历，现在正是世界观可以重塑的时候。
- 看某一本书以后，你突然觉得三观尽毁；过去所有 **fact** 不会变，但它们会以一种重新组织的方式链接起来；下一次你会带上这个新的理解。
- 讲者觉得这很有意思：你们有无限时间、无限可能性，可以把自己 **蒸馏成一个 skill**。

- 他再说失败很重要：从失败过程中，可以蒸馏出慢慢越来越成功的 skill；他称这就是他自己蒸馏出来的、关于自己的 skill。
- 讲者把这一段归到“用好大模型：遵循指令的能力”。

用好大模型：先会遵循指令，再处理真正 long horizon

- 现在已经知道遵循指令了，就可以看怎样处理真正 long horizon 的任务。
- 例子：计算机系统基础的 PA 作业。讲者认为，有明确 workflow 的可能都不算真正 long horizon。
- 真正 long horizon 多多少少带一点未知：从一个初始状态出发，你大概有一个 intent，但 specification 和 implementation 都不太清楚；最终目标是交付高质量的 implementation。
- 因此讲者说，不应该让 AI 一下子“画一下子”、一下子“向后坐坐”（原话此处模糊；（整理者按上下文理解）指不要让 AI 一次性急着包办/结束）。
- 原因：模型有 self-attention，反馈信号由 强化学习 给，所以它有一种着急的执念：赶快通过 workflow 满足你的要求就结束。
- 但它也有平衡：如果做的 planning 太多太长，会做不完，上下文满了，人会不满意。
- 所以讲者说：现阶段它最好还是一个“帮我把 PA2 做了”的状态。（整理者串联）这等于先让 AI 完成具体、可交付的一段，而不是直接包办未知的长程任务。

第 23 部分



Long-Horizon 能力是怎么“练”出来的？

模型是训练出来的

只要有反馈信号，就可以强化学习：答对了给奖励，答错了揍。后训练逼迫模型规划、思考，学会非常离谱的长程代码能力，能不断试错、解决问题，直到满足要求：连刚才那些折磨人的“语文”指令也顺便学会了。

- ◆ Long-horizon 不是一次性想得特别深，而是闭环控制：尝试 → 编译/测试 → 观察 → 修复。真正变长的是“出错以后还能回来”。
- ◆ 代码世界格外适合这件事：构建、测试、benchmark 都能反复给出便宜的判分信号，很像 AlphaGo Zero
- 拥有规则确定、结果可验证的棋盘。(Nature, 2017)

反馈也可能教坏模型

- ◆ 如果测试只覆盖表面行为，“坚持”也可能变成 reward hacking：不是逼近你的真实目标，而是把公开分数刷满。因此还需要 held-out tests 和独立 review。

现阶段：让 AI 做 PA2，而不是做长 planning

- 讲者认为，如果让 AI 做的 planning 太多、太长，结果是：它会做不完 → 上下文满了 → 人人会不满意。

- 所以在目前阶段，最好的用法还是让它“帮我把 **PA2** 做了”；讲者认为这是它最擅长的事情，做的人也会非常开心。
- 逻辑链（讲者给出）：
 1. planning 太多太长 → 做不完；
 2. 做不完 → 上下文满；
 3. 上下文满/任务失败 → 人人不满意；
 4. 所以现阶段应选 AI 最擅长的局部任务，例如 PA2。
- （整理者串联）这里的选择标准是：不让 AI 承担长程规划，而是把它放在当前能力最强、能交付的任务上。

从零做产品不难，难在往后维护：AI 会从某个点开始翻车

- 讲者在 **Codex**（原词：codex）里跑了一把，观察到：从零开始做产品看起来不难。
- 做一个小的 **proof of concept**，AI 马上就能给出来；个人主页也好、产品也好，听众应该都试过。
- 例子：**极品飞车 one shot**——只要说“给我一个极品飞车”，它就真的能跑。
- 但当你把它往后维护、加功能时，会观察到 AI 从某个时间点开始**翻车**；随着某个时间点开始，它的 **cost** 会越来越大。
- 讲者说这在软件工程里已经发生过无数次，**《人月神话》**讲的就是这件事。
- （整理者串联）即：从零生成一个能跑的 demo 很容易 ≠ 后续持续加功能的维护成本可控。

悖论：大型复杂软件系统里 AI 反而“砍瓜切菜”

- 讲者提出一个很有意思的悖论：一方面，小项目后续维护会翻车；另一方面，在大型、复杂的软件系统里，AI 却像**砍瓜切菜**。
- 典型例子是 **Linux 内核**。讲者说自己很幸运，在零几年、Linux 还处于 **2.6、甚至 2.6.2x** 的时代接触内核；那时源代码他还看得懂。
- 今天打开 Linux 内核源代码，立刻就是**劝退**：比如想学文件怎么打开，第一个是函数调用，点进去又是函数调用，点了十层，最后是一个 (x=y)，人马上就崩溃。
- 讲者解释：大型软件系统经过长时间演化，为了能维护下去，被迫变成现在这个样子。
- 结果恰好是：在大型复杂软件系统里，AI 反而游刃有余。

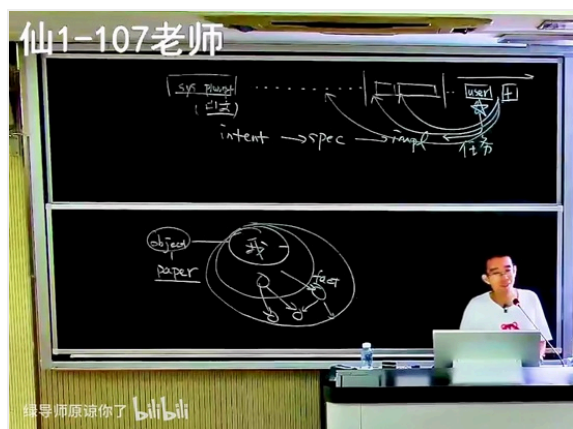
为什么？高质量人类维护代码提供“好上下文”，与论文污染上下文相反

- 讲者把这一点和前面的上下文污染对照：论文放进去后，会把上下文整个扭过去。

- 但如果是一个由人类维护的、特别高质量的代码库，上下文会迅速让 AI 看到里面好的东西；它会照着写、照着学。
- 例如要写一个功能，这个功能很可能在内核里已经在几个地方实现过类似逻辑。
- AI 的**无敌注意力机制**能马上找到那几个地方怎么实现：应该调哪个 **helper**（原词：help），按照什么顺序**上锁**。
- 这些对人类是巨大的心智负担，尤其在不了解那个 **subsystem** 时；你根本不知道该上什么锁。
- 但 **self-attention** 可以在加载几十个文件后看出：这几个地方都是这样写的，那就把它们加起来。
- 所以，越是在 Linux 内核这类大型系统中，AI 越能利用已有高质量范例。
- 讲者也提到另一段经历：前面 hack **Android Runtime**、改其中 **just-in-time (JIT)** 编译器时，这是非常非常痛苦的事情。

小结： 讲者用从零 PoC、后续维护、论文污染上下文、Linux 内核和 Android Runtime JIT 等例子说明，AI 的表现取决于上下文质量和已有范例，而不是简单由系统大小决定。

第 24 部分



大型系统，反而更容易？

Greenfield 悖论

“从零开始做产品看起来不难，AI 却会从某个时间点开始各种翻车。反而在大型复杂软件系统里，AI 游刃有余，令人迅速滑跪？”

- self-attention 可以“看到”好的 practice，并且立即 in-context learn，学到应该遵循的已有 best practice。
- 如果产品起步时没有建立良好的工程纪律，AI 会学会在 slop 上无限叠加。
- 启发：“提示词”不仅是你输入的，甚至代码库本身也是“提示词”。

代码少，不代表信息少

- ◆ 成熟仓库用命名、分层、错误处理和测试，把巨大的设计空间压成“照着邻居补全”：greenfield 里每个选择都悬空，信息熵反而更高。
- ◆ 模型只会追求局部一致性，并不知道眼前的是 best practice 还是历史事故。第一桶 slop 一旦成为多数派，后面的 slop 会像复利一样增长。

2/7

人类改 JIT 编译器极痛苦，AI 的复杂任务能力却会翻车

- 讲者回忆前面 hack **Android Runtime**、改 **JIT (just-in-time)** 编译器：非常非常痛苦。
- “为了改编译器死掉的同学”不少——最终决定不再做这个方向，甚至不愿意再写代码；讲者觉得太恐怖。

- AI 的对比：转录作“但 AI 就特别特别特别擅长让他做一个小游戏就可以，就会翻车”；按上下文应理解为：做小游戏可以，复杂一点就会翻车。
- （整理者串联）所以这里讨论的是 AI 的能力边界：小游戏与大型复杂改动不是同一难度。

prompt engineering / 上下文工程依赖当前上下文与模型结构，换模型就可能过时

- 讲者记得今天讲 prompt engineering / 上下文工程。
- 这些都是上下文和今天的模型结构带来的，存在逻辑推理关系。
- 新一代模型发布后，今天上课所有东西可能又没用了。
- 到时要重新想：新模型怎么训练、它的 personality 是什么、怎样更好地和它相处。

讲者认为长期不变的一点：模型不会开倒车

- 讲者判断：无论什么模型，在大型、复杂、维护良好的软件系统里面，它永远可以看到它该看到的东西，并且做对。
- 理由：模型永远可以满足指定/指令；听指令是不可能开倒车的。
- 所以讲者认为这点可以相信它。
- 逻辑链：
 1. 模型能听指令、满足指定。
 2. 听指令不会开倒车。
 3. 因此在大型、复杂、维护良好的软件系统中，模型能看到该看的并做对。
 4. 因而可以信任模型。

一个未经实验的“歪招”：把人类好设计经验作为上下文喂给模型

- 讲者自称有个歪招，没实验过，但觉得可能是好的 research。
- 设想：收集人类经验，例如人类历史上最好的 20 个项目；或每个领域找最好的 10/20 个项目。
- 从中提取好的设计经验。
- 例子：第一节课拿的 The Philosophy of Software Design。那就是人提炼出来的经验：类结构该怎么写、什么时候封装、什么时候做成继承关系。

- 这些是“碎碎念”：来自某些系统里这样写，且符合需求所以好。
- 操作：把这些经验切片切出来；甚至以污染上下文的形式，先把一个相近的好代码放进来，再放需求。
- 讲者的赌注：赌 **self-attention** 总能在某个 token 看到你，从而满足这个需求，而不是做那个；虽然污染上下文，但会把好的经验带过来。
- 目的：可能让 **AI slop**（转录作“AI slap”）延缓一点，更大概率做出可以长期维护的东西。讲者说“我不知道”。

这个歪招还需微调/精简，而且不同设计会打架

- 也许需要比较好的微调。
- 项目不必把所有代码都放进去；能不能像 **The Philosophy of Software Design** 那样做精简/总结，把好的模式提取出来，以文档形式放进去。
- 但讲者说这其实没那么容易：不同设计会打架。
- 例子：你的软件、我的软件都可以设计成完全松耦合、以发消息的方式交互。

第 25 部分

仙1-107老师

导师原谅你了 bilibili

“放一个好项目给 AI 看”靠谱吗？

调研结论：猜测成立一半

- 我的猜测：在代码库里放一个相近领域、优雅成熟的高质量系统，先让 AI 读一下，再实现需求，能得到质量更高的软件。
- ♦ 机制上合理，但“相近、精选、按需取用”比“整个仓库塞进去”重要。
- ♦ **RepoCoder** 中，迭代检索仓库相关片段比只看当前文件的 baseline 提升超过 10%；这支持“好示例有用”。
- ♦ 却没有证明外来大仓库必然有益。
- ♦ **CodeRAG-Bench** 的结论更克制：高质量 context 会帮忙，但“找对片段”和“真的用好片段”仍是瓶颈。
- ♦ 整库输入还可能带来 negative transfer：技术栈、约束、依赖或架构哲学不匹配；它也会消耗 context，让真正相关的约束被淹没。

一个能做的对照实验

- ♦ 固定任务、模型、token budget 和随机种子，对比：无示例 / 完整参考仓库 / 3-5 个精选 golden paths。
- ♦ 除了通过率，还要盲测架构违规、修改扩散面、review 时间、后续缺陷和代码克隆率。我的预测：精选的

不同设计会打架：不能同时要松耦合消息和统一 gateway 查表

- 讲者说，把不同设计合在一起其实没那么容易，因为不同设计会冲突。
- 例：你的软件和我的软件可以设计成完全松耦合地发消息；也可以设计成另一种：有一个统一的 **gateway**，把所有函数统一在里面，然后查表。

- 讲者判断：这两种设计不能同时有，只能选一个。
- 逻辑链：
 1. 设计 A=松耦合消息。
 2. 设计 B=统一 gateway+函数集中+查表。
 3. A 与 B 互斥，只能选一个。
 4. （整理者串联）所以“把多个软件设计都放进来”不是简单叠加，必须消解互斥、选出或收敛出一个结构。

可能方向：把多个设计交给 AI 多轮迭代，收敛到好结构再入库

- 讲者提出：也许更好的方法是把好几个软件的设计都放进来，然后由 AI 多轮迭代，收敛到一个好的结构，放到我们的代码库里。
- 讲者认为：这可能是一个还不错的方向；而且这个方向是新的。
- 与之前看的“A 问题拆成 A1/A2，在 A2 里做某种排列组合”相比，讲者觉得这个更有趣。

课程核心：没有超能力 AI，人仍要有概念、懂上下文

- 讲者把这一点称为这节课的核心观点：我们今天还没有这种超能力的 AI。
- 所以人还是需要有概念：
 - 知道上下文里面有什么；
 - 知道上下文里的每一个东西会对 AI 产生什么结果。
- 讲者说，达到这一点的唯一途径是：第一，你要对你玩的东西有概念。
- 因此仍然要学处理器、操作系统、编译器、数据库；讲者认为这里面有很多设计的智慧。
- 变化不在“要不要学”，而在“怎么学”：你会用更高效的方法来学。
- （整理者串联）这条链是：AI 不够超能力 → 人必须判断上下文 → 判断依赖概念 → 概念仍来自系统知识，只是学习方式改变。

用软件工程方法给指令、监控 AI、定义好的 verify

- 讲者说，理解上下文之后，会用一个学期解释：
 - 怎么用软件工程里的各种方法给出正确的指令；

- 怎么监控 AI 运行；
- 怎么给 AI 好的 **verify**；
- 什么叫好的 **verify**。
- 今天只提到：要在上下文里放进 **VERIF**。
- 至于“什么是好的 **VERIF**”，讲者没有给出定论；他说你们可以有自己的想法，他今天有一些比较松散的想法。
- 今天仍然需要把模型引导到你认为合适的位置上。

引导模型比组工程团队容易：提示词要持续可见

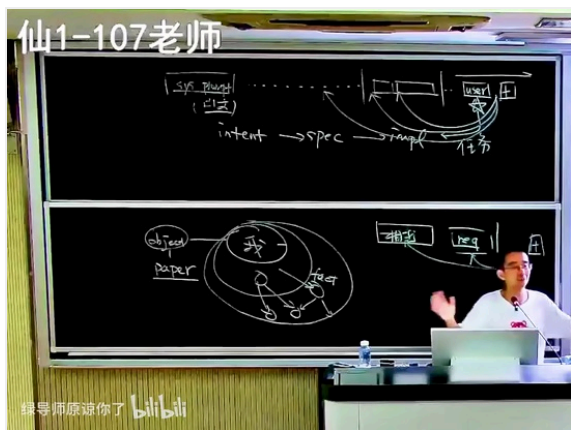
- 讲者对比：这已经比组一个工程团队容易。组团队要雇很多人，还要和他们“拉通对齐”（讲者称之为互联网黑话）。
- 讲者说：你不需要模型很聪明；你只需要把**正确、简练的提示词**放进上下文。
- 关键是让这个提示词**一直能被看到**，从而把模型往那个方向引导。
- （整理者串联）这里暗含的取向是：不靠模型自己变聪明，而靠人在上下文中持续给方向。

案例：符号执行引擎优化，一个“没有超能力 AI”的典型研究项目

- 讲者讲了一个“很好玩的故事”，作为没有超能力 AI 的非常典型案例。
- 项目是做一个**符号执行引擎的优化**。讲者说不需要知道这是什么；大概是要写一个程序，可以自动分析/证明/验证一个不算很大的程序的正确性。
- 现状问题：这个执行效率比较低，希望提出一些比较有效的优化。
- 讲者评价：这是非常合理的 **incremental** 题目，适合学生做。
- 原始 idea 大意是：用这种方式可能可以有效。
- 但有些东西是 open 的：这个东西到底怎么做、什么东西有效。
- 因此可以写一个 **research**。
- 这位同学很开心，认为这个 idea 有道理。

小结：讲者在本段反复落回同一点——AI 不是超能力，关键仍是人把上下文、引导和 **verify** 设计好。

第 26 部分



Research Failure: Token 花完了，知识没有增加

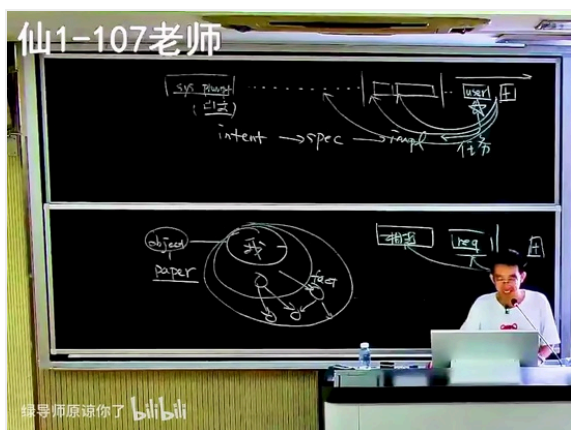
符号执行引擎优化

试图优化一个符号执行引擎：有一个原始 idea，问题带一点 active、open-ended research 的味道，但也只是希望往前推进一点点。

- PhD student: GPT-5.6 Sol Ultra 启动！benchmark 丢进去，想法丢进去，花了很多 token，没有结果。

问题可能出在哪？

- 缺的不是 token，而是便宜、局部、可归因的反馈。一个总 benchmark 分数几乎不给下一步方向。
- “没变快”可能来自 path explosion、solver、缓存、内存、搜索策略、实现 bug 或环境噪声；Agent 不知道
- 该强化哪一步。
- 更要命的是：Agent 被训练成“把任务做完”，研究却允许最好的结论是“原始 idea 不成立”。没有可证伪假设，它可能把错误想法实现得越来越完整。
- 经典的 KLEE 把问题拆成搜索策略、约束缓存、independent constraint sets 等机制，再分别测量。
- 更合理的起点：先 profile 主导瓶颈，把 idea 改写成一句可证伪预测，造几十秒就有反馈的



Research Failure: Token 花完了，知识没有增加

符号执行引擎优化

试图优化一个符号执行引擎：有一个原始 idea，问题带一点 active、open-ended research 的味道，但也只是希望往前推进一点点。

- PhD student: GPT-5.6 Sol Ultra 启动！benchmark 丢进去，想法丢进去，花了很多 token，没有结果。

问题可能出在哪？

- 缺的不是 token，而是便宜、局部、可归因的反馈。一个总 benchmark 分数几乎不给下一步方向。
- “没变快”可能来自 path explosion、solver、缓存、内存、搜索策略、实现 bug 或环境噪声；Agent 不知道
- 该强化哪一步。
- 更要命的是：Agent 被训练成“把任务做完”，研究却允许最好的结论是“原始 idea 不成立”。没有可证伪假设，它可能把错误想法实现得越来越完整。
- 经典的 KLEE 把问题拆成搜索策略、约束缓存、independent constraint sets 等机制，再分别测量。
- 更合理的起点：先 profile 主导瓶颈，把 idea 改写成一句可证伪预测，造几十秒就有反馈的

“证明 A 方法有用”这句 prompt 把模型变成 reward hacker

- 同学认为 idea 有道理；GPT5.6SALA（原文如此）也认为这个 idea 有道理，于是开始跑，消耗了非常多的 token。
- 同学的困惑是：GPT 不断给他反馈，但他无法验证反馈正确与否；当 GPT 给 negative feedback 时，他也不知道该怎么打破。
- 讲者用“原汤化原食”说这个失败：这个失败就是 GPT5.6 生成的，错误犯在上下文中的一句话上。
- 那句话是：“我想证明 A 方法是有用的。”
- 讲者给出的逻辑链：
 1. 把“证明 A 方法有用”设成目标；
 2. 模型看到这句话后，忘记了更大的 research big picture；

- 3. 模型因此变成一个 **reward hacker**;
 - 4. 它构造很多小 **benchmark**, 在小 benchmark 上做微小的东西, 试图给出正面或负面结论;
 - 5. 这些结论对真正的研究问题没有用。
- 所以讲者认为: 这是方法上的错误, 不是单纯模型能力不够。

纠正: 从“小 benchmark 上证明有用”转向真实工具与 SOTA

- 讲者认为, 作为审稿人时, 人可以站在人类立场上评审论文; 但作为构建者, 不能反过来这样要求模型直接证明方法有用。
- 构建者只能一点一点从小处开始做, 逐步证明方法有用。
- 同学做错的一步是: **太想证明 idea 有用**, 而这正是 prompt 诱导出来的。
- 纠正方案是: 讲者说“我告诉你 PH14 (原文如此, 具体含义待核)”, 也就是应从**真实的工具**和 **state-of-the-art 算法**开始做 benchmark。
- 为什么这样做值得: 哪怕要跑几个小时、会 **timeout**, timeout 中产生的**搜索路径**也很有价值; 它揭示了什么样的**程序结构**对现有探索方法是不好的。
- 讲者评价: 这是一个好玩的 **research failure**, 也说明“今天人还是有用的”。

人仍有用: 没有概念就提不出正确问题

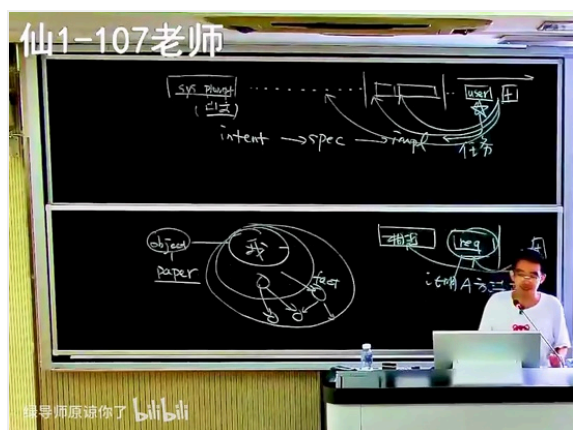
- 讲者承认, 没有办法把所有事情都交给 AAI (原文如此; 按上下文指 AI/AGI 一类系统)。
- 但与此同时, 工作方式要发生变化。
- 第一, 这些知识很重要: **操作系统、编译器等**。
 - 如果没有概念, 就提不出正确的问题;
 - 找不到引导的正确方向;
 - 对人类知识没有概念, 也很难引导模型解决一个 **open question**。
- 所以人的作用至少包括: 提出正确问题、找方向、注入领域知识。

与 AI 协作的 trade-off: 盯着纠正 vs 给更多 token

- 讲者的观察：如果盯着 Codex 的 session，它一旦跑偏就立即纠正，可以更快完成任务。
- 例如实现某个东西，比如今天要上交互系统干个什么事；如果纠正它，把自己的领域知识注入进去，总时间可能更快。
- 但如果浪费更多 token、给它更多时间，它一般来说也能完成。
- 于是出现 **trade-off**：到底花自己的钱/人力去盯着纠正，还是花 token/时间让 AI 自己跑。讲者的话在这里停在这个权衡上。

小结：错误目标会诱导 reward hacking；纠正方向是真实工具、SOTA 算法与可解释的失败路径，人的领域知识仍用于提问和引导。

第 27 部分



少干预，反而做得更多

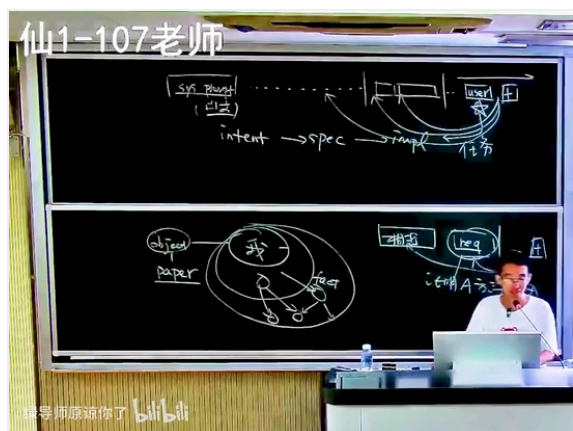
Latency 和 Throughput 是两回事

一般来说，我对要解决的问题了解更多，所以我干预，会完成得更快。但是我越来越少干预了：AI 走弯路也能完成，那我还不如切换到另一个任务。只要结果能满足，我更倾向于不管。

- ◆ 人类干预优化单个任务的 latency：放手让 Agent 跑，优化的是一整天的 throughput。机器多走一点弯路，换回最稀缺的人类注意力。
- ◆ 放手成立有两个前提：任务可以并行，失败可以便宜地恢复。否则“少干预”只是把故障推迟。

Review 什么？

- 比如接口我 review 过了，实现是 AI slop，最后出了问题了修就行 😊
- ◆ 按“不可逆性 × 爆炸半径”分配注意力：接口、不变量、数据格式、权限和测试早看；可替换的叶子实现可以晚看。
- ◆ 丰田的“自働化”也是这个逻辑：一个人照看多台机器，靠异常自动停机和 andon 报警，而不是盯住每一步。Agent 的对应物是 CI、预算、权限边界和 stop condition。([Toyota Production System](#))



模型是**训练**出来的，尤其是只是有反馈信号，就可以强化学习（学对了给奖励，学错了给惩罚）。逼近大模型规划、思考，在后训练阶段进行学会非常离谱的长程代码能力（可以想象成是一个很长的 cot），能不断试错、解决问题，直到满足——甚至把指令逻辑顺便一起解决了。刚才的“语文”指令也能理解了（虽然没有那么多折磨人的语文资料）。

一个有趣的发现：

从开始做产品（看起来不难），AI 就从某个时间点开始各种翻车。但反而在大型复杂软件系统里，AI 游刃有余，令人迅速滑跪？

因为 self-attention 可以“看到”好的 practice 并且立即 in-context learn 学到（上下文压缩 = 降智）应该遵循已有的 best practice，但如果产品起步时没有建立良好的工程纪律，AI 会学会在 slop 上无限叠加。启发：“提示词”不仅是你输入的，甚至代码库本身也是“提示词”。

我的猜测（没有做过实验）：在你的代码库里放一个相近领域、优雅成熟的高质量系统，先让 AI 读一下代码，再实现你的需求，能得到质量更高的软件。[[调研一下我的猜测合理不合理。]]

对你要实现的东西**有概念**，现在仍然是非常重要的。Sadly，这只能通过**学习**获得，没有“超能力”。

AI“”，会用一学期来解释如何用软件工程的方法：给出正确指令，监控 AI 运行，防止 AI 失控、---

62/lect/index.md 116,8-1 75%

更多 token/时间通常能换来完成，但人力有限，于是出现 trade-off

讲者说：如果浪费更多 token、给 AI 更多时间，它一般也能完成。可这就产生一个 trade-off：token 要花钱，而每个同学的人力、精力是有限的。因此每个同学都该思考，要把自己宝贵的精力放到“怎么处理 AI”上。

逻辑链：

1. 给更多 token/时间 → AI 一般能完成。
2. 但 token 花钱、人的精力有限。
3. 所以必须权衡：精力应投到收益最大的事情上。

（整理者串联）如果精力总量固定，把精力花在低收益的 AI 互动上，就会挤掉高收益工作。

纠正 AI 有多巴胺：讲者认为它会制造“我仍比 AI 有用”的心理安慰

讲者自我检讨：纠正 AI 是一件“有多巴胺”的事。它给的感觉是：虽然 AI 很强，我好像还是比 AI 有点用。这会产生非常微妙的心理安慰。

相对应的是另一种模式：**YOLO 模式**（原文作“yo mod”）——让 AI 自己搞，等它回来；自己不看历史，只验收；如果真搞定了，就不看了。讲者认为，这种“放手一验收”可能在未来很长时间里成为同学们工作的常态。

把时间留给高收益、真正必须学生做的事；已知怎么做且 AI 能做好的事就放手

讲者说，只有把 AI 用在把自己的时间留在收益最大的事情上，这部分时间必须留给高收益事情。对不知道、还不清楚怎么弄的事情，这几乎应是学生 100% 的时间要做的事；剩下“你应该怎么干、AI 也能干好”的事，就放手让 AI 干。

这就像 pre-AI 时代已经讨论过的 **假努力 vs 有效努力**：自我感动式的努力没有用，要有效努力。AI 时代效果差距拉得更大：你每一秒钟的假努力，如果有一个 **peer** 在真努力，你的损失非常大。讲者称这是自己的小心得。

长上下文下指令遵循下降，但最后一条具体指令仍常可遵循，所以“干活”仍好用

讲者说，随着指令、上下文增长，模型的指令遵循能力确实会下降。但现在模型有个很有意思的特点：一般最后那条 prompt / 最后的指令还是可以遵循的；越往前、尤其到中间，越不遵循上下文指令。

因此干活还是很好用，因为最后一个短窗口通常就是一个具体任务：“帮我修什么东西”。它真的还能把这件事做完。所以模型非常值得使用。

真正 long-horizon、在不确定性中探索的任务，ChatGPT 解决不了

讲者区分：真正 long-horizon 的非常难任务，不是解决一个问题，而是在不确定性中探索。这类任务 ChatGPT 解决不了。

ChatGPT 能解决的是有确定解法/算法、能确定地一直往后推进的问题。（原文音近“有VER法”，识别不清）

让 AI 选“最佳方案”容易得到非最佳：模型被训练成完成任务优先的“舔狗”

如果你让 ChatGPT “考虑几种方案，哪个最好”，它很可能给出一个不是最好的方案。讲者认为原因：模型已经被训练成“舔狗”，它完成任务才有奖励，完不成任务没有奖励，所以会给你一个有限的方案，而不是最优方案。

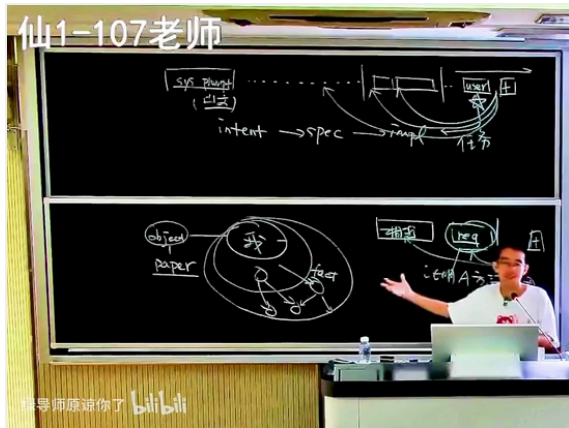
P/NP 类比：验证一个解容易，找到满足条件的解难

讲者用 P/NP 复杂度类作类比：验证一个解比较容易，但找到满足条件的解很难。

（整理者串联）这正对应前面的担忧：AI 可以接受并验证一个已给出的方案，但让它自己找到“最好”的解，并不轻松。

小结：讲者把 AI 使用看成精力配置问题——把最后一条具体、确定的任务交给 AI 执行，把高不确定性探索和最优判断留给自己。

第 28 部分



在不确定性中探索

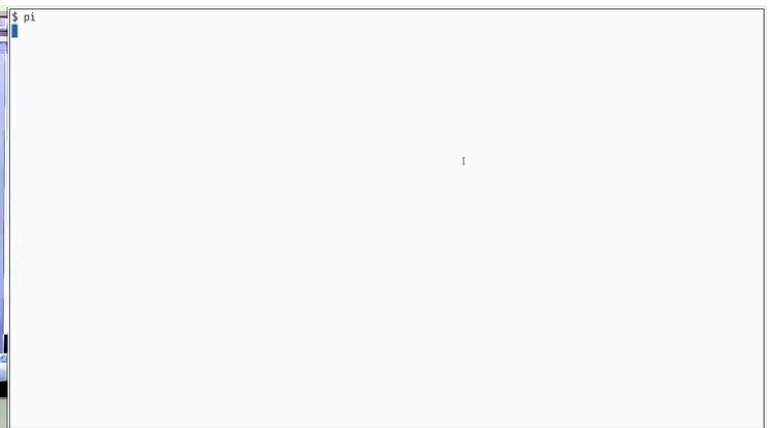
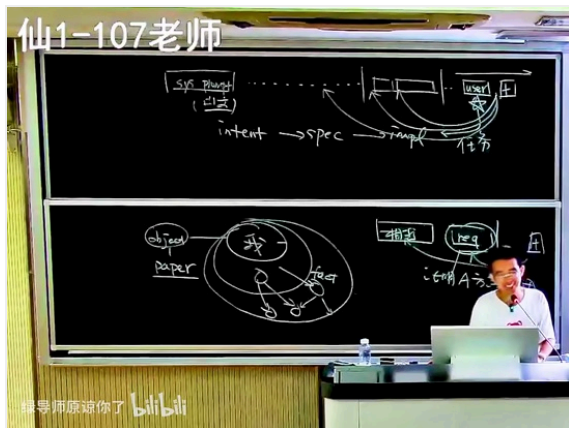
这次不是“解题”

我就是希望模型在不确定性中探索。

- 如果所有要求和路线都想清楚了，这就是一个相对无趣的问题：Chain-of-thought 就能解决
- 今天“帮我考虑一下几种方案”有效，但可能不是最佳的
- 模型被训练成了“完成任务”才有奖励，所以它会给你有**限**的方案：complexity gap 里，那个“更好”的可能被藏起来了

谁规定“已经想完了”？

- ◆ “给我几种”只规定了停止条件 k ，没有规定覆盖率。这正是 Herbert Simon 所说的**满意化**：搜到“够好”就停。
- ◆ CoT 用算力换**深度**：沿一条路多走几步；并行枚举用算力换**宽度**：让更多轨迹进入低概率的尾部。
- ◆ 探索的瓶颈常常不是“想不出”，而是模型太早认定“已经够了”。



逻辑链（整理者串联）：验证容易、搜索难 → 只能有限搜索 → 直接 next token 趋同 → 系统性拆分换智力 → 个体 bias 汇成 $1+1>2$

1. NP 式任务：验证一个解容易，找到满足条件的解难。
2. 因此，模型为了完成任务，势必只能做有限搜索。
3. 如果直接让 AI 用 next token 生成，它会把自身的偏见带进结果，起名等开放任务会趋同。
4. 替代方案是系统性调研/枚举，把大空间拆成更明确的小问题，用 subagent 检查，实现算力换智力；代价是烧 token。
5. 讲者进一步联想到人类社会：世界很大、人很小，个体 bias 不可消除，但正因如此才有 $1+1>2$ 。

验证容易 ≠ 找到答案容易：模型完成任务只能有限搜索

- 讲者用 **NP 问题** 说明：验证一个解比较容易（原话：“是 P 的”），但想找到满足条件的解很难。
- 例子：给你一条路径，判断它是不是 ****Hamilton path（哈密顿路径）**** 很容易；但要找到一个满足条件的 Hamilton path，就是 **NP complete（NP 完全问题）**。**3-SAT** 也属于这类完全问题。
- 从“搜索困难、验证容易”这个模型出发，讲者推断：模型为了完成任务，**势必只能做有限的搜索**。

考试类比：写比不写分高，所以有限努力也要出手

- 讲者用考试类比：在考试场上花点时间，尽量把题写完；马上要交卷时，**写比不写分数高**，所以一定要写点什么。
- 讲者认为，这时就可以使用一个**算力换智力的循环**。

DeepSeek 起名：模型品味提升，但直接生成制造“AI 时代子涵”

- 讲者观察：比听众再小五岁的班上，一个班可能有 5 个甚至 10 个“子涵”；现在子涵少多了。
- 他看 AI 起名，觉得“还是很好的”：AI 知道去《诗经》里找，模型的品味提升了。
- 但他“还是不敢用”。原因：你用 **DeepSeek** 起名，别人也用 DeepSeek 起名，大家会得到同样的名字，于是你成为“**AI 时代的子涵**”。（原文 ASR 作 deep pick/D4C/DIVSK）
- 讲者说，直接让 AI 用 **next token** 方式生成，它会给出“它的偏见带来的这个东西”。

破解同质化：系统性调研/枚举 + subagent 检查，把算力换成智力

- 讲者说，如果真需要一个非常 **[low horizon]**（原文如此）的任务，就需要一个**系统性的调研过程**。
- 类比科研实践课：进入一个科研领域，第一件事是**文献调研**，穷尽与这个问题相关的所有文献。
- 讲者判断：AI 想快速完成任务时，暂时还不会去花这个大代价。

- 但名字是可以枚举的：先选音，把不好的音筛掉；选完音，再用 **subagent** 一个一个检查字。比如喜欢《诗经》，就选《诗经》里的词。
- 这样就把开放问题变成可枚举、可检查的子任务，实现算力换智力。

拆分开放空间：切成更明确的小问题，更多算力 → 更大智力；代价烧 token

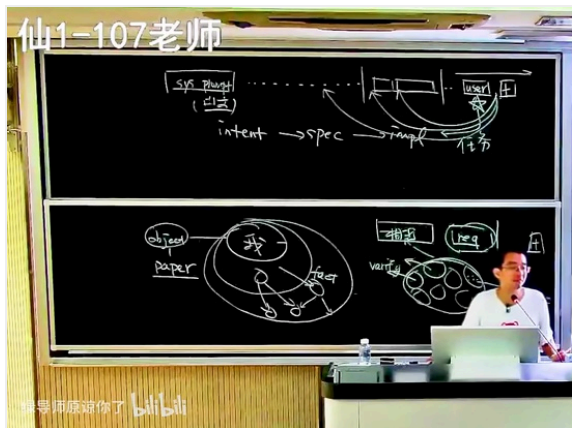
- 讲者说，开放问题在一个非常大的空间里。直接让 AI 用 next token，会得到受偏见影响的结果。
- 如果把空间分割成很多个 [**destroy**]（疑为 distinct）的部分，每个部分就会变成有 [**WIFI**]（疑为 well-defined）的更明确的东西。
- 讲者得出的结论是：有**更多算力**，就有**更大的智力**。
- 缺点：烧 token 头疼。

从个体 bias 到群体 $1+1>2$ ：世界很大、人很小，学不完但差异有价值

- 讲者由此谈对人类社会的一个理解：世界很大，人很小。
- 人类的品味每个人都不一样。因为我们带着很大的 **bias**（原文 ASR 作 BS），在不同城市、不同教育背景、不同生活圈长大。
- 讲者提到 **big world hypothesis**（原文前的人名 ASR 乱码，无法还原）：我们学不到这个世界上所有的东西。
- 但讲者认为，正因为带着这个 **bias**，才有 $1+1>2$ ；我们就是一个最大的群体。逻辑上：个体无法穷尽世界，各自有不同的局部视角和偏好，群体差异与协作反而可以超过单个个体之和。（整理者串联）

小结：讲者把 NP 式搜索困难推到 AI 任务策略：直接 next token 会趋同，系统性拆分加算力可换智力；而个体 bias 在群体层面反而成为 $1+1>2$ 的条件。

第 29 部分



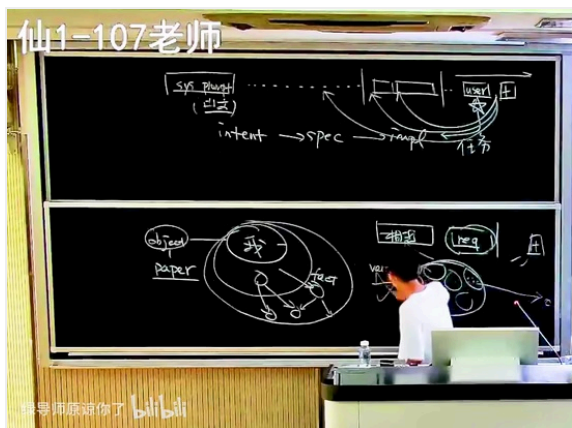
我们一直在“多采样”

世界很大，人很小

- 人的品味来源于 diversity: 不同的教育背景、生活圈、经历.....Richard Sutton's [Big World Hypothesis](#)
- 人天生对所有问题都有 bias; 但 $1 + 1 > 2$: 无论是和舍友菜鸡互啄，还是去开 conference，都是在理解对方的 bias, 促进群体智能，然后每个人继续带着 bias 创新



- 群体智能不要求每个人没有偏见；它只要求偏见不能只剩一种。交流也不是尽快共识，而是先让不同误差活下来。



墙内的超人



算力成为生产资料

- 这也是未来可能发生、非常残酷的现实：资本家可以获得无限的计算资源，获得无限的智力，“人”本身的价值在减弱——砌一堵墙，墙里面是超人，墙外面是动物世界
- Stanford AI Index 2025: 2024 年近 90% 的 notable models 来自产业界；若干前沿模型的训练算力成本已经是数千万到近两亿美元。

个体全知不可能，但 bias 让群体出现 $(1+1>2)$

- 讲者的前提：我们学不到世界上所有东西。
- 但正因为带着 bias（讲者原词；ASR 作 bias s），才有 $(1+1>2)$ 。
- 所以讲者说：我们就是一个最大的群体智能；我们在一起实现智能。

群体智能也带来未来危机：真正融合可能让“算力换智力”

- 讲者明确说，这会带来一个未来的危机。
- 他先说“我不知道”，但要把这个问题抛出来。
- 他的说法是：如果这种真正融合来讲，就是真正有价值；而真正有价值的问题，可以用算力换智力。

- 讲者没有展开“真正融合”具体指什么，也没有解释“算力换智力”的机制；他直接追问：这可能发生什么样的残酷现实？

残酷现实：无限算力 → 无限智力 → 墙内超人、墙外动物

讲者给出的链条是：

1. 有一类人可以获得**无限的计算资源**。
2. 因此他们拥有**无限的智力**。
3. 他们就可以砌一堵墙。
4. 墙里面的人有智力，是“我的朋友”；墙外面的人是普通人。
5. 假设计算机、AI 模型比人类智力更高效。
6. 那么墙外面的人就变成**动物园里的猴子**，被放任自生自灭。
7. 墙外人无论如何也赶不上墙内人产生**智力的速度**。
8. 结局：墙里面是**超人**，墙外面是**动物世界**。

AI 关键奇点临近，先要思考如何防止

- 讲者认为，大家已经在 AI 的**关键奇点**快要到的时候。
 - 因此需要思考：**怎样防止这件事到来**。
 - 此段中，讲者只提出危机与待防目标，没有给出具体防止方案。
-